GUÍA DE INDUCCIÓN · DESARROLLO EN RUST

Patrones, arquitectura y stack en Rust

Para quien llega al equipo desde otro lenguaje: Rust explicado paso a paso, los idioms y patrones del libro *Rust Design Patterns*, nuestra arquitectura limpia y una inducción por cada tecnología del stack, construyendo un servicio real.

Parte 0 · Plan de inducción y entorno

Parte 1 · Cartilla de sintaxis

Parte 2 · Idioms

Parte 3 · Patrones de diseño

Parte 4 · Antipatrones

Parte 5 · Nuestra arquitectura

Parte 6 · Inducciones: Cargo, errores, Tokio, validations_rs, PostgreSQL, axum, gRPC, Tauri, Dioxus

Parte 7 · Trabajar con EGM

Parte 8 · Ejercicios y proyecto final

Fuentes: *Rust Design Patterns*, documentación oficial de Tokio, axum, tokio-postgres, tonic, Tauri y Dioxus, el código fuente de cada versión y el video *Rust Data Modelling Without Classes* (No Boilerplate) · noviembre de 2026

Contenido

Cómo usar esta guía

Parte 0 · Plan de inducción

A quién está dirigida y cómo se recorre

Preparar el entorno

Parte 1 · Cartilla de sintaxis

1.1 Variables: let, mut y shadowing
1.2 Funciones, bloques y expresiones
1.3 Tipos básicos, String y &str
1.4 Structs, impl y métodos
1.5 Enums, match y patrones
1.6 Option, Result y el operador ?
1.7 Ownership y préstamos
1.8 Lifetimes: 'a
1.9 Traits, genéricos, impl Trait y dyn Trait
1.10 Closures
1.11 Módulos y visibilidad
1.12 Chuleta de símbolos

Parte 2 · Idioms

2.1 Recibí tipos prestados en los argumentos
2.2 Constructores: new y Default
2.3 mem::take y mem::replace para cambiar de variante sin clonar
2.4 Mutabilidad temporal
2.5 Devolvé el argumento consumido cuando hay error
2.6 Finalización en destructores: el finally de Rust
2.7 #[non_exhaustive]: cuándo NO usarlo
2.8 Option como iterador

Parte 3 · Patrones de diseño

3.1 Newtype
3.2 Builder
3.3 Strategy: inversión de dependencias
3.4 Command
3.5 RAII con guardas
3.6 Descomponé structs para préstamos independientes
3.7 Preferí crates chicos
3.8 Traits propios para simplificar bounds
3.9 Genéricos como type classes: dividir la API por tipo
3.10 Los otros patrones del libro

Parte 4 · Antipatrones

4.1 Clonar para conformar al borrow checker
4.2 Polimorfismo con Deref: la tentación de la herencia
4.3 #![deny(warnings)]

Parte 5 · Nuestra arquitectura

5.1 Las capas y quién depende de quién
5.2 Una responsabilidad por tipo
5.3 De Kotlin y Spring a nuestro stack
5.4 El núcleo, archivo por archivo
5.5 SOLID en Rust

Parte 6 · Inducciones por tecnología

6.1 Cargo y workspaces
6.2 Errores con thiserror y anyhow
6.3 Async con Tokio
6.4 Reglas de negocio con validations_rs 0.2
6.5 PostgreSQL con tokio-postgres y deadpool-postgres
6.6 REST con axum
6.7 gRPC con tonic y el crate proto
6.8 Escritorio y móvil con Tauri 2
6.9 Interfaces en Rust con Dioxus 0.7

Parte 7 · Trabajar con EGM

7.1 Instalar nuestro EGM
7.2 La idea: un elefante, peces y una persona
7.3 Qué agrega el perfil Rust
7.4 Un ciclo completo, con el proyecto de inducción
7.5 El contexto del proyecto

Parte 8 · Ejercicios y proyecto final

8.1 Ejercicios del lenguaje
8.2 Proyecto final de inducción

Fuentes

Cómo usar esta guía

Esta guía tiene dos mitades. La primera (Partes 1 a 4) enseña el lenguaje y la forma de diseñar en Rust, apoyada en el libro comunitario **Rust Design Patterns**, que divide el tema en tres grupos:

- **Idioms:** convenciones que la comunidad sigue al escribir código. Son las "normas sociales": las rompés solo si tenés una buena razón.
- **Patrones de diseño:** soluciones reutilizables a problemas que se repiten.
- **Antipatrones:** soluciones que parecen buenas pero traen más problemas de los que resuelven.

El libro hace una advertencia que vale para todo lo que sigue: **cada patrón tiene costos, y lo importante es entender por qué lo elegís, no solo cómo se implementa.** Rust no es orientado a objetos, así que muchos patrones clásicos cambian de forma o directamente desaparecen.

La segunda mitad (Partes 5 a 8) muestra cómo trabajamos: nuestra arquitectura, una inducción por cada tecnología del stack, el flujo de diseño con EGM y un proyecto final. Todo gira alrededor de un **proyecto de inducción** que se entrega junto con esta guía.

La **Parte 1** es una cartilla que podés consultar en cualquier momento. En el resto de la guía, cada construcción nueva se explica en una caja de **Sintaxis**, y las cajas **Desde Kotlin / Java** comparan con lenguajes de la JVM.

Cómo se verificó el código. Los programas completos de las Partes 1 a 4 (los que tienen `fn main`) se compilaron y ejecutaron con Rust 1.97; podés copiarlos en el Rust Playground. Las capas `domain` y `application` del proyecto de inducción compilan y pasan sus tests y clippy. El código de axum, tonic, tokio-postgres, Tauri y Dioxus se escribió contra el código fuente de cada versión; el primer paso de la inducción es compilarlo con `cargo check`.

Parte 0 · Plan de inducción

A quién está dirigida y cómo se recorre

Esta guía es para quien llega al equipo desde otro lenguaje (Kotlin, Java, C#, TypeScript, Go, Python) y tiene que desarrollar en nuestros proyectos Rust. No hace falta saber Rust de antes. Al terminarla vas a poder leer y escribir código en nuestra arquitectura, con nuestro stack, y trabajar con nuestro flujo de diseño (EGM).

Hay un **proyecto de inducción** que acompaña toda la guía: un servicio de registro de usuarios con las capas `domain`, `application`, `proto` e `infrastructure`, que expone el mismo caso de uso por REST (axum) y por gRPC (tonic) y guarda en PostgreSQL. Cada módulo de la Parte 6 explica una pieza de ese proyecto.

Etapa	Qué leer	Qué hacer	Checkpoint
1. Entorno	Sección 0.2	Instalar Rust, el editor y las herramientas.	`cargo test` pasa en el proyecto de inducción (capas `domain` y `application`).
2. El lenguaje	Partes 1 y 2	Escribir y correr los ejemplos en el Rust Playground.	Ejercicios 1 y 2 de la Parte 8.
3. Diseño en Rust	Partes 3 y 4	Reconocer cada patrón y antipatrón en el código del proyecto.	Ejercicios 3 a 7 de la Parte 8.
4. Nuestra arquitectura	Parte 5	Recorrer las capas del proyecto de inducción.	Explicar a otra persona qué crate puede depender de cuál y por qué.
5. Backend	Parte 6, módulos 6.1 a 6.7	Levantar el servicio con PostgreSQL y llamarlo por REST y gRPC.	Checkpoint de cada módulo.
6. Frontend (según el rol)	Parte 6, módulos 6.8 y 6.9	Crear una app Tauri o Dioxus que llame a un caso de uso.	Checkpoint de cada módulo.
7. Flujo de trabajo	Parte 7	Diseñar una funcionalidad chica con EGM.	Un diseño aprobado por `/egm-review`.
8. Proyecto final	Parte 8	Agregar un caso de uso completo al proyecto de inducción.	PR revisado por el equipo.

Las cajas **Sintaxis** explican cada construcción nueva de Rust apenas aparece. Las cajas **Desde Kotlin / Java** comparan con lenguajes de la JVM; si venís de otro lenguaje, la idea es la misma (por ejemplo, una `sealed class` de Kotlin equivale a una unión discriminada de TypeScript).

Preparar el entorno

Rust. Instalalo con `rustup`, que maneja las versiones del compilador:

- **Windows:** descargá `rustup-init.exe` desde rustup.rs. Necesita las herramientas de compilación de C++ de Visual Studio (el instalador te avisa si faltan).
- **macOS y Linux:** `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`

El perfil por defecto ya trae `cargo` (compilador y gestor de paquetes), `rustfmt` (formateador) y `clippy` (linter). Comprobalo con `cargo --version` y `cargo clippy --version`.

Editor. VS Code (o un editor compatible) con la extensión **rust-analyzer**: autocompletado, tipos inferidos en línea y errores mientras escribís. En JetBrains, RustRover cumple el mismo rol.

Herramientas del stack, a medida que llegues a cada módulo:

Herramienta	Para qué	Cómo se instala
Docker	PostgreSQL local (6.5)	Docker Desktop o el paquete de tu sistema.
`protoc`	Generar código gRPC (6.7)	Windows: el `.zip` de protobuf releases agregado al `PATH`; macOS: `brew install protobuf`; Linux: `apt install protobuf-compiler`.
`grpcurl`	Probar servicios gRPC a mano (6.7)	Desde grpcurl releases.
Prerrequisitos de Tauri	Apps de escritorio y móvil (6.8)	Guía de prerrequisitos de Tauri (en Windows: WebView2 y las herramientas de C++).
`dx` (Dioxus CLI)	Apps Dioxus (6.9)	`cargo install dioxus-cli` y `rustup target add wasm32-unknown-unknown`; `dx doctor` verifica la instalación.
EGM	Diseño y revisión (Parte 7)	Ver la sección 7.1.

Comandos que vas a usar todos los días:

```
cargo build                 compila
cargo run -p infrastructure ejecuta el binario de un crate del workspace
cargo test                  corre todos los tests (unitarios, de integración y doctests)
cargo fmt                   formatea el código
cargo clippy --all-targets --all-features -- -D warnings   linter, como en CI
cargo doc --open            genera y abre la documentación, incluida la de las dependencias
```

`cargo doc --open` merece una mención: genera la documentación de **todas** tus dependencias en la versión exacta que usás, sin depender de internet. Es la forma más confiable de consultar una API.

Parte 1 · Cartilla de sintaxis

1.1 Variables: `let`, `mut` y shadowing

```rust
fn main() {
    let edad = 30;           // inmutable: como `val` en Kotlin
    let mut contador = 0;    // mutable: como `var`
    contador += 1;

    let precio: f64 = 99.5;  // tipo anotado con `nombre: Tipo`

    let texto = "42";
    let texto: i32 = texto.parse().unwrap(); // shadowing: una variable NUEVA con el mismo nombre

    println!("{edad} {contador} {precio} {texto}");
}
```

SINTAXIS

- `let` declara; sin `mut`, la variable no se puede reasignar ni modificar.
- El tipo casi siempre se infiere. Cuando lo escribís, va después de dos puntos, igual que en Kotlin.
- **Shadowing:** volver a escribir `let texto` crea otra variable que tapa a la anterior, incluso con otro tipo. Es muy común para "transformar" un valor sin inventar nombres (`texto_str`, `texto_num`).
- `println!` lleva `!` porque es una **macro**, no una función. `{edad}` interpola la variable; `{}` toma el siguiente argumento.
- Las líneas terminan en `;`. En Rust el punto y coma tiene significado (lo vemos enseguida).

1.2 Funciones, bloques y expresiones

En Rust casi todo es una **expresión** (produce un valor). Esto explica una regla que confunde al principio: **la última línea de un bloque, sin `;`, es el valor del bloque**.

```rust
fn doble(x: i32) -> i32 {
    x * 2 // sin `;`: este es el valor de retorno
}

fn clasificar(n: i32) -> &'static str {
    if n < 0 {
        "negativo"
    } else if n == 0 {
        "cero"
    } else {
        "positivo"
    }
}

fn main() {
    let y = {
        let a = 3;
        a + 1 // el bloque entero vale 4
    };
    println!("{} {} {}", doble(y), clasificar(-5), clasificar(y));
}
```

SINTAXIS

- `fn nombre(param: Tipo) -> TipoRetorno { ... }`. Si no hay `->`, la función devuelve `()` (el "unit", como `Unit` en Kotlin).
- `x * 2` sin punto y coma es el resultado. Si escribieras `x * 2;`, la expresión se convierte en sentencia, el bloque devuelve `()` y el compilador se queja de que esperaba un `i32`.
- `return x;` existe, pero se usa solo para salir antes de tiempo.
- `if` es una expresión, como en Kotlin: `let signo = if n < 0 { "-" } else { "+" };`.
- `&'static str` es "una referencia a texto que vive todo el programa" (los literales entre comillas). El `'static` es un *lifetime*; lo vemos en la sección 1.8.

1.3 Tipos básicos, `String` y `&str`

Rust	Kotlin	Notas
`i8` `i16` `i32` `i64` `i128`	`Byte` `Short` `Int` `Long`	Enteros con signo
`u8` `u16` `u32` `u64`	`UByte` … `ULong`	Sin signo; muy usados
`usize`	(no existe)	Tamaño de puntero; se usa para índices y longitudes
`f32` `f64`	`Float` `Double`	
`bool`, `char`	`Boolean`, `Char`	`char` es Unicode (4 bytes)
`String`	`String` (o `StringBuilder`)	Texto propio y modificable, en el heap
`&str`	—	Préstamo de un texto que pertenece a otro
`Vec<T>`	`MutableList<T>`	Lista que crece
`&[T]`	—	Préstamo de una porción de lista (*slice*)
`(i32, &str)`	`Pair<Int, String>`	Tuplas de cualquier largo

Rust	Kotlin	Notas
`()`	`Unit`	

```rust
fn main() {
    let literal: &str = "hola"; // texto fijo, guardado en el binario
    let mut propio: String = String::from("hola");
    propio.push_str(" mundo");
    let vista: &str = &propio[0..4]; // préstamo de una parte

    let numeros: Vec<i32> = vec![1, 2, 3];
    let par: (i32, &str) = (1, "uno");
    let (n, nombre) = par; // desestructuración de la tupla

    println!("{literal} | {propio} | {vista} | {numeros:?} | {n} {nombre}");
}
```

SINTAXIS

- **`String` vs `&str`** es la distinción que más se repite. `String` es *dueño* del texto. `&str` es una *vista prestada* a un texto que es de otro (un `String`, un literal, un pedazo). Regla práctica: los campos de tus structs suelen ser `String`; los parámetros de funciones que solo leen, `&str`.
- `&propio[0..4]` toma los bytes del 0 al 3. `0..4` es un rango que excluye el final; `0..=4` lo incluye.
- `vec![...]` es una macro que crea un `Vec`.
- `{numeros:?}` usa el formato **Debug** (para desarrolladores), que requiere que el tipo implemente `Debug`. `{}` usa **Display** (para usuarios finales).

1.4 Structs, `impl` y métodos

```rust
#[derive(Debug, Clone, PartialEq)]
struct Punto {
    x: i32,
    y: i32,
}

struct Metros(f64); // tuple struct: campos sin nombre
struct Marcador;    // unit struct: sin datos

impl Punto {
    // Función asociada: no recibe `self`. Se llama con `::`
    fn origen() -> Self {
        Self { x: 0, y: 0 }
    }

    // Método que solo lee: `&self`
    fn manhattan(&self) -> i32 {
        self.x.abs() + self.y.abs()
    }

    // Método que modifica: `&mut self`
    fn mover(&mut self, dx: i32) {
        self.x += dx;
    }

    // Método que consume: `self` (el valor deja de existir para quien lo llamó)
    fn en_tupla(self) -> (i32, i32) {
        (self.x, self.y)
    }
}

fn main() {
    let mut p = Punto::origen();
    p.mover(3);
    println!("{:?} distancia={}", p, p.manhattan());

    let q = Punto { y: 7, ..p.clone() }; // "copy" con cambios
    println!("{q:?}");

    let m = Metros(1.5);
    println!("{} metros", m.0); // los campos de un tuple struct se acceden por posición
    let _ = Marcador;

    let t = p.en_tupla();
    // println!("{p:?}"); // no compila: `p` fue consumido por `en_tupla`
    println!("{t:?}");
}
```

SINTAXIS

- Los datos (`struct`) y el comportamiento (`impl`) se declaran por separado. Podés tener varios bloques `impl` para el mismo tipo.
- `Self` (con mayúscula) es "el tipo que estoy implementando". `self` (minúscula) es la instancia.
- El primer parámetro define qué hace el método con la instancia: `&self` la **lee**, `&mut self` la **modifica**, `self` la **consume** (se la queda).
- `Tipo::funcion()` para funciones asociadas (el equivalente a un `companion object`); `valor.metodo()` para métodos.
- `Punto { y: 7, ..p.clone() }` es la *struct update syntax*: los campos que no nombrás se toman de otro valor. Es el `copy(y = 7)` de Kotlin.
- `#[derive(...)]` le pide al compilador que genere implementaciones automáticas: `Debug` (imprimir con `{:?}`), `Clone` (método `.clone()`), `PartialEq` (`==`). Es lo que una `data class` te da gratis.

1.5 Enums, `match` y patrones

```rust
#[derive(Debug)]
enum Pago {
    Efectivo,
    Tarjeta { ultimos4: String, cuotas: u8 },
    Transferencia(String), // el alias de destino
}

fn describir(p: &Pago) -> String {
    match p {
        Pago::Efectivo => "efectivo".to_string(),
        Pago::Tarjeta { cuotas: 1, .. } => "tarjeta en un pago".to_string(),
        Pago::Tarjeta { ultimos4, cuotas } if *cuotas > 12 => {
            format!("tarjeta {ultimos4}: demasiadas cuotas")
        }
        Pago::Tarjeta { ultimos4, cuotas } => format!("tarjeta {ultimos4} en {cuotas} cuotas"),
        Pago::Transferencia(alias) => format!("transferencia a {alias}"),
    }
}

fn main() {
    let pagos = vec![
        Pago::Efectivo,
        Pago::Tarjeta { ultimos4: "4242".into(), cuotas: 3 },
        Pago::Transferencia("mi.alias".into()),
    ];
    for p in &pagos {
        println!("{}", describir(p));
    }

    // `if let`: cuando solo te interesa una variante
    if let Pago::Transferencia(alias) = &pagos[2] {
        println!("alias: {alias}");
    }

    // `let ... else`: desestructurar o salir
    let Pago::Tarjeta { cuotas, .. } = &pagos[1] else {
        return;
    };
    println!("cuotas: {cuotas}");
}
```

SINTAXIS

- Cada variante puede no tener datos (`Efectivo`), tener campos con nombre (`Tarjeta { .. }`) o campos posicionales (`Transferencia(String)`).
- `match` es exhaustivo y gana el **primer brazo que coincide**, por eso el caso `cuotas: 1` va antes del general.
- Dentro de un patrón: `..` ignora el resto de los campos, `_` ignora un valor, `|` combina alternativas (`1 | 2 => ...`), `13..=19` es un rango, e `if condición` después del patrón es una **guarda**.
- Como hacemos `match` sobre una referencia (`&Pago`), los campos se enlazan también como referencias: `cuotas` es `&u8` . Por eso la guarda usa `*cuotas` : el `*` **desreferencia** (accede al valor apuntado).
- `"4242".into()` convierte un `&str` en `String` (el tipo destino se infiere del campo).
- `format!` es como `println!` pero devuelve un `String` .

DESDE KOTLIN / JAVA

```kotlin
sealed interface Pago {
    data object Efectivo : Pago
    data class Tarjeta(val ultimos4: String, val cuotas: Int) : Pago
    data class Transferencia(val alias: String) : Pago
}
```

Un `enum` de Rust es tu `sealed interface` , y `match` es tu `when` exhaustivo. La diferencia es que en Rust las variantes **no son tipos propios**: `Pago::Tarjeta` no es un tipo que puedas recibir como parámetro, solo el `Pago` completo.

1.6 `Option` , `Result` y el operador `?`

Rust no tiene `null` ni excepciones. Los dos enums más importantes de la librería estándar los reemplazan:

```rust
use std::num::ParseIntError;

// Result<T, E>: Ok(T) si salió bien, Err(E) si falló
fn parsear_edad(s: &str) -> Result<u8, ParseIntError> {
    let n: u8 = s.trim().parse()?; // si `parse` falla, retorna el Err acá mismo
    Ok(n)
}

// Option<T>: Some(T) si hay valor, None si no
fn posicion(ids: &[u32], buscado: u32) -> Option<usize> {
    ids.iter().position(|&id| id == buscado)
}

fn main() {
    match parsear_edad("42") {
        Ok(e) => println!("edad {e}"),
        Err(e) => println!("error: {e}"),
    }

    let pos = posicion(&[10, 20, 30], 20).map(|i| i + 1).unwrap_or(0);
    let edad = parsear_edad("abc").unwrap_or_default(); // 0 si falla
    let obligatorio: Result<usize, &str> = posicion(&[1], 1).ok_or("no encontrado");

    println!("{pos} {edad} {obligatorio:?}");
}
```

SINTAXIS

- `?` al final de una expresión: si es `Err` (o `None`), **retorna** ese error desde la función actual; si es `Ok(v)` (o `Some(v)`), lo **desenvuelve** y sigue con `v`. Solo funciona dentro de funciones que devuelven `Result` (u `Option`).
- `.unwrap()` saca el valor o **hace panic** (el programa aborta). `.expect("mensaje")` es lo mismo con un mensaje. Usalos en tests y prototipos, no en lógica de negocio.
- `|&id| id == buscado` es una **closure** (lambda). El `&id` desestructura la referencia que entrega el iterador.

Kotlin	Rust
`T?`	`Option<T>`
`x?.let { f(it) }`	`x.map(f)`
`x ?: valorPorDefecto`	`x.unwrap_or(valor)`
`x!!`	`x.unwrap()`
`throw + try/catch`	devolver `Err(e)` + `match` o `?`
`x ?: return`	`let Some(v) = x else { return }`

1.7 Ownership y préstamos

Esta es la idea que hace a Rust distinto de cualquier lenguaje con recolector de basura. Tres reglas:

1. Cada valor tiene **un único dueño**.
2. Cuando el dueño sale de alcance, el valor se libera (no hay garbage collector).
3. Podés **prestar** el valor: muchas referencias de lectura (`&T`) **o** una sola de escritura (`&mut T`), nunca las dos a la vez.

```rust
fn consumir(s: String) -> usize { s.len() } // recibe el valor: ahora es su dueño
fn leer(s: &str) -> usize { s.len() }       // préstamo de lectura
fn agregar(s: &mut String) { s.push('!'); } // préstamo de escritura

fn main() {
    let a = String::from("hola");
    let b = a; // MOVE: el dueño ahora es `b`; `a` ya no se puede usar
    // println!("{a}"); // no compila: "borrow of moved value"

    let n = leer(&b);      // `&b` presta `b`
    let mut c = b.clone(); // copia explícita: otro String independiente
    agregar(&mut c);       // préstamo exclusivo
    let m = consumir(c);   // `c` se mueve a la función

    let x = 5;
    let y = x; // los enteros son `Copy`: se copian solos y `x` sigue siendo válida

    println!("{n} {m} {x} {y} {b}");
}
```

SINTAXIS

- `&valor` crea un préstamo de lectura; `&mut valor` uno de escritura (la variable tiene que ser `mut`).
- En un tipo, `&T` y `&mut T` son "referencia a T".
- `*referencia` accede al valor apuntado (casi siempre es automático al llamar métodos).
- Los tipos simples (`i32`, `bool`, `char`, tuplas de ellos) implementan `Copy`: se duplican al asignarlos. Todo lo demás se **mueve**, salvo que llames a `.clone()`.

DESDE KOTLIN / JAVA

En Kotlin y Java, `val b = a` copia una *referencia* y los dos nombres apuntan al mismo objeto; el GC decide cuándo liberarlo. En Rust, `let b = a` **transfiere la propiedad**. Esto elimina en compilación los data races y los use-after-free, y es la razón de que aparezcan `&`, `&mut`, `clone()` y `Arc` por todos lados.

1.8 Lifetimes: `'a`

Un lifetime es un nombre para "cuánto tiempo vive un préstamo". El compilador los infiere casi siempre, pero a veces necesita que se lo digas:

```rust
// "El resultado vive tanto como el más corto de los dos préstamos"
fn mas_largo<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

// Un struct que guarda un préstamo necesita declarar su lifetime
struct Vista<'a> {
    texto: &'a str,
}

impl<'a> Vista<'a> {
    fn primera_palabra(&self) -> &'a str {
        self.texto.split(' ').next().unwrap_or("")
    }
}

fn main() {
    let frase = String::from("hola mundo");
    let v = Vista { texto: &frase };
    println!("{} | {}", mas_largo("uno", "tres"), v.primera_palabra());
}
```

SINTAXIS

- `<'a>` declara un parámetro de lifetime, igual que `<T>` declara un tipo genérico. Se lee "a".
- `&'a str` es "un préstamo de `str` que vive al menos `'a`".
- `'static` es el lifetime especial de lo que vive todo el programa.
- `impl<'a> Vista<'a>` repite la declaración: "para cualquier lifetime `'a`, implemento `Vista<'a>`". Lo mismo pasa con genéricos: `impl<T> Caja<T>`.
- En aplicaciones de negocio, la mayoría de los structs son **dueños** de sus datos (`String`, no `&str`) justamente para no tener que lidiar con lifetimes.

1.9 Traits, genéricos, `impl Trait` y `dyn Trait`

```rust
use std::fmt::Display;
use std::sync::Arc;

trait Saludo {
    fn nombre(&self) -> String;

    // Método con implementación por defecto
    fn saludar(&self) -> String {
        format!("Hola, {}", self.nombre())
    }
}

struct Persona { nombre: String }
impl Saludo for Persona {
    fn nombre(&self) -> String { self.nombre.clone() }
}

struct Robot;
impl Saludo for Robot {
    fn nombre(&self) -> String { "R2".into() }
    fn saludar(&self) -> String { "BIP BOP".into() } // reemplaza el default
}

// Genérico con restricción (bound): despacho estático
fn presentar<T: Saludo>(x: &T) -> String { x.saludar() }

// Lo mismo, con sintaxis corta
fn presentar_corto(x: &impl Saludo) -> String { x.saludar() }

// `where` para restricciones largas
fn juntar<T, U>(t: T, u: U) -> String
where
    T: Display,
    U: Display + Clone,
{
    format!("{t} y {u}")
}

// Devolver "algo que implementa Saludo" sin nombrar el tipo concreto
fn fabricar() -> impl Saludo { Robot }

fn main() {
    let p = Persona { nombre: "Mati".into() };
    println!("{}", presentar(&p));
    println!("{}", presentar_corto(&Robot));
    println!("{}", juntar(1, "dos"));
    println!("{}", fabricar().saludar());

    // Despacho dinámico: tipos distintos en la misma colección
    let todos: Vec<Box<dyn Saludo>> = vec![Box::new(p), Box::new(Robot)];
    for s in &todos {
        println!("{}", s.saludar());
    }

    // Arc: puntero compartido con contador de referencias, seguro entre hilos
    let compartido: Arc<dyn Saludo + Send + Sync> = Arc::new(Robot);
    let otro = Arc::clone(&compartido);
    println!("{} referencias a {}", Arc::strong_count(&otro), otro.nombre());
}
```

SINTAXIS

- `trait` es una interfaz; `impl Trait for Tipo { ... }` la implementa. A diferencia de Kotlin, **podés implementar tus traits para tipos ajenos** (por ejemplo, `impl Saludo for i32`).
- `<T: Saludo>` se lee "cualquier tipo `T` que implemente `Saludo`". El compilador genera una copia de la función por cada tipo concreto (*monomorfización*): es rápido, sin costo en runtime.
- `T: Display + Clone` exige varios traits a la vez.
- `impl Trait` en un parámetro es azúcar para un genérico. En el retorno, significa "un tipo concreto que no te digo".
- `dyn Trait` es un **objeto trait**: el método se resuelve en runtime (como una interfaz de Java). Como su tamaño no se conoce en compilación, siempre va detrás de un puntero: `&dyn Trait`, `Box<dyn Trait>` o `Arc<dyn Trait>`.
- `Box<T>` guarda el valor en el heap con un único dueño. `Arc<T>` lo comparte entre varios dueños; `Arc::clone` **no copia el dato**, solo incrementa el contador.
- `Send + Sync` son traits marcadores: "se puede mover a otro hilo" y "se puede compartir entre hilos".

Una pieza más que vas a ver seguido: los **tipos asociados**. Un trait puede declarar un tipo que cada implementación elige:

```
trait Repositorio {
    type Id; // cada implementación decide qué tipo es
    fn existe(&self, id: &Self::Id) -> bool;
}

struct RepoDeUsuarios;
impl Repositorio for RepoDeUsuarios {
    type Id = u64;
    fn existe(&self, id: &u64) -> bool { *id < 100 }
}

fn main() {
    println!("{}", RepoDeUsuarios.existe(&7));
}
```

El ejemplo más famoso es `Iterator`, que declara `type Item;`: cada iterador dice qué elementos produce.

1.10 Closures

```rust
fn aplicar<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 {
    f(x)
}

fn main() {
    let factor = 3;
    let por_factor = |x| x * factor; // captura `factor` prestado

    let mut total = 0;
    let mut acumular = |x: i32| total += x; // modifica lo que captura
    acumular(5);
    acumular(7);

    let nombre = String::from("Ferris");
    let saludo = move || format!("hola {nombre}"); // `move`: se lleva `nombre`

    println!("{} {} {}", aplicar(por_factor, 2), total, saludo());

    let cuadrados_pares: Vec<i32> = (1..=10)
        .filter(|n| n % 2 == 0)
        .map(|n| n * n)
        .collect();
    println!("{cuadrados_pares:?}");
}
```

SINTAXIS

- `|parámetros| cuerpo`. Con varias líneas: `|x| { ...; resultado }`.
- Hay tres traits para closures, según qué hacen con lo que capturan: `Fn` (solo lee), `FnMut` (modifica), `FnOnce` (consume; se puede llamar una sola vez).
- `move` fuerza a la closure a **adueñarse** de lo que captura. Es obligatorio cuando la closure va a vivir más que la función que la creó (hilos, tareas async).
- `.collect()` transforma un iterador en una colección; el tipo destino sale de la anotación (`Vec<i32>`). Si no hay anotación, usás el *turbofish*: `.collect::<Vec<_>>()`.

1.11 Módulos y visibilidad

```rust
mod dominio {
    pub mod valores {
        pub struct Email(String); // el tipo es público, el campo NO

        impl Email {
            pub fn parse(s: &str) -> Option<Self> {
                s.contains('@').then(|| Email(s.to_string()))
            }
            pub fn as_str(&self) -> &str {
                &self.0
            }
        }
    }

    pub(crate) fn interno() -> &'static str {
        "visible en todo el crate, invisible afuera"
    }
}

use dominio::valores::Email;

fn main() {
    let e = Email::parse("a@b.com").unwrap();
    // let trucho = dominio::valores::Email("x".into()); // no compila: campo privado
    println!("{} / {}", e.as_str(), dominio::interno());
}
```

SINTAXIS

- Todo es **privado por defecto**. `pub` lo hace público; `pub(crate)` lo limita al crate actual (parecido a `internal` de Kotlin).
- La privacidad es **por módulo**, no por tipo: dentro de `mod valores` cualquier código ve el campo `.0`; fuera, nadie.
- `use ruta::Item;` importa. `crate::` es la raíz del crate actual, `self::` el módulo actual y `super::` el padre.
- `::` separa rutas (módulos, tipos, funciones asociadas); `.` accede a campos y métodos de un valor.
- `condición.then(|| valor)` devuelve `Some(valor)` si es `true` y `None` si es `false`.

1.12 Chuleta de símbolos

Símbolo	Significado	Ejemplo
`::`	Ruta: módulo, tipo, función asociada, variante	`Pago::Efectivo`, `String::new()`
`.`	Campo o método de un valor	`p.x`, `s.len()`
`&` / `&mut`	Préstamo de lectura / de escritura	`leer(&b)`, `agregar(&mut c)`
`*`	Desreferenciar	`*contador += 1`
`'a`	Lifetime	`&'a str`, `'static`
`?`	Propagar error o `None`	`let n = s.parse()?;`

Símbolo	Significado	Ejemplo
`!` al final	Invocación de macro	`println!`, `vec!`, `matches!`
`#[...]`	Atributo del ítem siguiente	`#[derive(Debug)]`, `#[test]`
`#![...]`	Atributo del módulo o crate que lo contiene	`#![allow(dead_code)]`
`|x| ...`	Closure	`.map(|n| n * 2)`
`->`	Tipo de retorno	`fn f() -> i32`
`=>`	Brazo de un `match`	`None => 0`
`..`	Rango, o "el resto" en un patrón o struct	`0..10`, `Tarjeta { .. }`
`_`	Comodín: ignorar o "inferí el tipo"	`_ => {}`, `Vec<_>`
`<T>`	Parámetro genérico	`fn f<T: Clone>(x: T)`
`::<T>`	Turbofish: tipo explícito al llamar	`"5".parse::<i32>()`
`Self` / `self`	El tipo implementado / la instancia	`fn new() -> Self`
`r#"..."#`	String crudo (sin escapes)	`r#"{"a":1}"#`

Parte 2 · Idioms

2.1 Recibí tipos prestados en los argumentos

El libro recomienda preferir siempre el **tipo prestado** antes que un préstamo del tipo dueño: `&str` antes que `&String`, `&[T]` antes que `&Vec<T>`, y `&T` antes que `&Box<T>`.

```rust
fn tiene_tres_vocales(palabra: &str) -> bool {
    let mut seguidas = 0;
    for c in palabra.chars() {
        match c {
            'a' | 'e' | 'i' | 'o' | 'u' => {
                seguidas += 1;
                if seguidas >= 3 {
                    return true;
                }
            }
            _ => seguidas = 0,
        }
    }
    false
}

fn suma(numeros: &[i32]) -> i32 {
    numeros.iter().sum()
}

fn main() {
    let propio = String::from("guiaba");
    println!("{}", tiene_tres_vocales(&propio)); // &String se convierte solo en &str
    println!("{}", tiene_tres_vocales("Ferris")); // un literal ya es &str

    // `split` produce &str: con un parámetro &String esto no compilaría
    for palabra in "el cangrejo guiaba la nave".split(' ') {
        if tiene_tres_vocales(palabra) {
            println!("{palabra} tiene tres vocales seguidas");
        }
    }

    let v = vec![1, 2, 3];
    println!("{} {}", suma(&v), suma(&[10, 20])); // sirve para Vec, arrays y slices
}
```

Por qué: un `&String` solo acepta referencias a un `String` existente. Un `&str` acepta literales, `String`, partes de un `String` y todo lo que produzca texto prestado. Además, `String` ya es un puntero a datos en el heap, así que `&String` es un puntero a un puntero: una indirección de más.

SINTAXIS: DEREF COERCION

El compilador convierte automáticamente `&String` en `&str`, `&Vec<T>` en `&[T]` y `&Box<T>` en `&T`. Esto se llama *deref coercion* y funciona en un solo sentido: de lo dueño a lo prestado.

En nuestra arquitectura: por eso los constructores de los value objects reciben `&str` (`Email::try_from(raw: &str)` en la Parte 5).

2.2 Constructores: `new` y `Default`

Rust no tiene constructores como construcción del lenguaje. La convención es una función asociada llamada `new`. Para los valores por defecto existe el trait `Default`, que se puede derivar si todos los campos lo implementan.

```rust
#[derive(Debug, Default, PartialEq)]
struct ConfigServidor {
    host: String,               // por defecto: ""
    puerto: u16,                // 0
    tls: bool,                  // false
    origenes_cors: Vec<String>, // []
    timeout_seg: Option<u64>,   // None
}

impl ConfigServidor {
    fn new() -> Self {
        Self {
            host: "0.0.0.0".into(),
            puerto: 8080,
            ..Default::default() // el resto, con sus valores por defecto
        }
    }
}

fn main() {
    let base = ConfigServidor::new();
    let con_tls = ConfigServidor { tls: true, ..ConfigServidor::new() };
    let vacia = ConfigServidor::default();

    println!("{base:#?}");
    assert_ne!(base, con_tls);
    assert_eq!(vacia.puerto, 0);
}
```

SINTAXIS

- `..Default::default()` completa los campos que no escribiste. Es la *struct update syntax* de la sección 1.4.
- `{base:#?}` imprime el Debug con saltos de línea (*pretty print*).
- `assert_eq!` y `assert_ne!` son macros que hacen panic si no se cumple la comparación. Se usan en tests, pero funcionan en cualquier lugar.
- También podés implementar `Default` a mano: `impl Default for ConfigServidor { fn default() -> Self { ... } }`.

El libro aclara dos cosas: es común y esperado que un tipo tenga **las dos cosas**, `new()` y `Default`; y la ventaja de `Default` es que tu tipo funciona con métodos genéricos de la librería estándar, como `unwrap_or_default()` o `mem::take` (que vemos a continuación).

DESDE KOTLIN / JAVA

En Kotlin escribirías `data class ConfigServidor(val host: String = "0.0.0.0", val puerto: Int = 8080, ...)`. Rust **no tiene parámetros con valores por defecto ni parámetros con nombre**, así que el trabajo lo reparten `Default`, la struct update syntax y el patrón Builder (sección 3.2).

2.3 `mem::take` y `mem::replace` para cambiar de variante sin clonar

Este idiom es exclusivo de Rust y conecta con el video: cambiar el estado de una entidad modelada como enum.

El problema: tenés un `&mut Entrada` en estado `Disponible { codigo }` y querés pasarla a `Reservada { codigo, usuario }`. No podés "sacar" el `codigo` de una referencia prestada, porque el borrow checker exige que **siempre haya algo** en ese lugar. La salida fácil sería `codigo.clone()`, pero eso es un antipatrón (sección 4.1) y aloca memoria de más.

`mem::take` saca el valor y deja en su lugar el valor por defecto del tipo (para `String`, un string vacío, que no aloca):

```rust
use std::mem;

#[derive(Debug)]
enum Entrada {
    Disponible { codigo: String },
    Reservada { codigo: String, usuario: u32 },
}

impl Entrada {
    fn reservar(&mut self, usuario: u32) -> Result<(), String> {
        match self {
            Entrada::Disponible { codigo } => {
                // Sacamos `codigo` (queda un "" temporal) y reemplazamos la variante entera
                *self = Entrada::Reservada { codigo: mem::take(codigo), usuario };
                Ok(())
            }
            otra => Err(format!("no se puede reservar: {otra:?}")),
        }
    }
}

fn main() {
    let mut e = Entrada::Disponible { codigo: "A-0001".into() };
    e.reservar(7).unwrap();
    println!("{e:?}");
    println!("{:?}", e.reservar(8)); // ya estaba reservada
}
```

SINTAXIS

- `match self` sobre un `&mut Entrada` enlaza los campos como `&mut String`. Por eso `codigo` se puede pasar a `mem::take`, que recibe `&mut T`.
- `*self = ...` reemplaza el valor completo al que apunta la referencia.
- `otra => ...` es un patrón que captura cualquier cosa con un nombre (como `else ->` en un `when` de Kotlin, pero con el valor disponible).
- `mem::replace(codigo, String::from("x"))` hace lo mismo pero te deja elegir qué poner en el lugar. Sirve cuando el tipo no implementa `Default`.
- Para `Option`, existe el atajo `opcion.take()`: devuelve el contenido y deja `None`.

El libro lo compara con Indiana Jones: para llevarte el ídolo, tenés que dejar una bolsa de arena en su lugar.

2.4 Mutabilidad temporal

Muchas veces un dato se prepara (se ordena, se filtra) y después solo se lee. Podés dejar esa intención explícita volviéndolo inmutable:

```rust
fn obtener_precios() -> Vec<u64> {
    vec![3000, 1500, 4500]
}

fn main() {
    // Opción 1: bloque anidado
    let precios = {
        let mut p = obtener_precios();
        p.sort();
        p
    };

    // Opción 2: rebinding
    let mut nombres = vec!["Zoe", "Ana", "Luz"];
    nombres.sort();
    let nombres = nombres; // desde acá, inmutable

    // precios.push(1); // no compila: `precios` no es mut
    println!("{precios:?} {nombres:?}");
}
```

Es un detalle chico, pero el compilador te garantiza que nadie modifica los datos después de cierto punto.

2.5 Devolvé el argumento consumido cuando hay error

Si una función que puede fallar **consume** un argumento, devolvelo dentro del error. Así quien llama puede reintentar sin haber clonado el valor por las dudas. La librería estándar lo hace, por ejemplo, en `String::from_utf8`: si los bytes no son UTF-8 válido, el error te devuelve el vector original.

Combinado con lo que vimos, sirve para transiciones de estado **por valor**:

```rust
#[derive(Debug)]
enum Entrada {
    Reservada { codigo: String, usuario: u32 },
    Pagada { codigo: String, usuario: u32, pago_id: String },
}

impl Entrada {
    // Consume la entrada. Si no se puede pagar, la devuelve junto con el motivo.
    fn pagar(self, pago_id: String) -> Result<Entrada, (Entrada, &'static str)> {
        match self {
            Entrada::Reservada { codigo, usuario } => {
                Ok(Entrada::Pagada { codigo, usuario, pago_id })
            }
            otra => Err((otra, "solo se puede pagar una entrada reservada")),
        }
    }
}

fn main() {
    let e = Entrada::Reservada { codigo: "A-0001".into(), usuario: 7 };
    let e = e.pagar("pago-123".into()).unwrap();
    println!("{e:?}");

    match e.pagar("pago-456".into()) {
        Ok(_) => println!("pagada otra vez (no debería pasar)"),
        Err((original, motivo)) => println!("{motivo}; la entrada sigue igual: {original:?}"),
    }
}
```

Fijate que en `pagar` no hay ni un `clone()`: `codigo` y `usuario` se **mueven** de la variante vieja a la nueva. Comparado con la versión `&mut self` + `mem::take`, esta es más directa; la otra es útil cuando la entidad vive dentro de otra estructura y solo tenés una referencia.

SINTAXIS

- `Result<Entrada, (Entrada, &'static str)>`: el tipo del error es una **tupla**. En el `match`, `Err((original, motivo))` la desestructura.
- `Ok(_)` ignora el contenido.

2.6 Finalización en destructores: el `finally` de Rust

Rust no tiene `finally`. Para ejecutar código "pase lo que pase" al salir de una función (retorno normal, `?` o panic) se usa el destructor de un objeto, implementando el trait `Drop`:

```rust
use std::time::Instant;

struct Cronometro {
    etiqueta: &'static str,
    inicio: Instant,
}

impl Drop for Cronometro {
    fn drop(&mut self) {
        println!("[{}] tardó {:?}", self.etiqueta, self.inicio.elapsed());
    }
}

fn dividir(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 { Err("división por cero".into()) } else { Ok(a / b) }
}

fn procesar(n: i32, d: i32) -> Result<i32, String> {
    let _medir = Cronometro { etiqueta: "procesar", inicio: Instant::now() };
    if n < 0 {
        return Err("negativo".into()); // el destructor corre igual
    }
    let r = dividir(n, d)?; // y también si `?` sale antes
    Ok(r)
}

fn main() {
    println!("{:?}", procesar(10, 2));
    println!("{:?}", procesar(-1, 2));
    println!("{:?}", procesar(10, 0));
}
```

Detalles que el libro remarca:

- La variable tiene que llamarse `_medir` (con nombre). Si la llamás `_` a secas, el valor se destruye **en esa misma línea**.
- Los destructores corren casi siempre, pero no están garantizados (por ejemplo, si el proceso aborta). No los uses para algo que *absolutamente* tiene que ejecutarse.
- Es código implícito: leyendo la función no se ve qué corre al final. Documentalo.

Este idiom es la base del patrón **RAII** (sección 3.5).

DESDE KOTLIN / JAVA

Es la idea de `use { }` sobre un `Closeable` o de `try-with-resources` en Java, pero sin bloque especial: el alcance de la variable ya define cuándo se cierra.

2.7 `#[non_exhaustive]`: cuándo NO usarlo

El atributo `#[non_exhaustive]` permite que el autor de una **librería** agregue variantes a un enum o campos a un struct sin romper a sus usuarios. El costo es que obliga a todos los que hacen `match` a escribir un brazo `_ =>` para "algo que todavía no existe".

```rust
mod pagos {
    #[non_exhaustive]
    #[derive(Debug)]
    pub enum Medio {
        Efectivo,
        Tarjeta,
    }
}

fn main() {
    let m = pagos::Medio::Efectivo;
    // Dentro del mismo crate el atributo no tiene efecto; entre crates,
    // el compilador exigiría además un brazo `_ => ...`.
    match m {
        pagos::Medio::Efectivo => println!("efectivo"),
        pagos::Medio::Tarjeta => println!("tarjeta"),
    }
}
```

En nuestra arquitectura: no lo pongas en los enums de nuestro propio workspace. Si `domain` marcara `DomainError` como `#[non_exhaustive]`, la capa de infraestructura (otro crate) tendría que agregar un `_ =>` al mapear errores a códigos HTTP, y perderías justo lo más valioso del `match` exhaustivo: que agregar una variante nueva **rompa la compilación** donde falta manejarla. El libro mismo dice que muchas veces es mejor subir la versión mayor que usar este atributo.

2.8 `Option` como iterador

Un `Option` se puede ver como una colección de cero o un elemento, así que funciona en cualquier lugar que acepte un iterador:

```rust
fn main() {
    let invitado: Option<&str> = Some("Turing");
    let mut asistentes = vec!["Curry", "Kleene"];
    asistentes.extend(invitado); // agrega solo si es Some

    let extra: Option<&str> = None;
    for nombre in asistentes.iter().chain(extra.iter()) {
        println!("{nombre}");
    }

    // filter_map: map que descarta los None
    let edades: Vec<u8> = ["30", "x", "41"].iter().filter_map(|s| s.parse().ok()).collect();
    println!("{edades:?}");
}
```

SINTAXIS

`.ok()` convierte un `Result<T, E>` en `Option<T>` (descartando el error). `filter_map` aplica la función y se queda solo con los `Some`.

Parte 3 · Patrones de diseño

3.1 Newtype

Qué es: un tuple struct de un solo campo que envuelve otro tipo para crear un tipo **nuevo e incompatible**. Es lo que usamos para los value objects (`Email`, `UserId`).

El libro destaca usos que vale la pena tener presentes:

```rust
use std::fmt;

// 1. Cambiar el comportamiento de un tipo existente: una contraseña que no se filtra en logs
struct Password(String);

impl fmt::Display for Password {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "********")
    }
}

impl fmt::Debug for Password {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Password(********)")
    }
}

impl Password {
    fn exponer(&self) -> &str { &self.0 } // acceso explícito y fácil de buscar
}

// 2. Distinguir identificadores y unidades
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct UsuarioId(u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct EventoId(u64);

fn cancelar(usuario: UsuarioId, evento: EventoId) {
    println!("cancelando {usuario:?} en {evento:?}");
}

// Un alias NO protege: es el mismo tipo con otro nombre
type Kilometros = f64;
type Millas = f64;

fn main() {
    let p = Password("hunter2".into());
    println!("{p} {p:?} (largo real: {})", p.exponer().len());

    cancelar(UsuarioId(7), EventoId(42));
    // cancelar(EventoId(42), UsuarioId(7)); // no compila: tipos invertidos

    let k: Kilometros = 10.0;
    let m: Millas = k; // compila sin quejas: ¡bug silencioso!
    println!("{m}");
}
```

SINTAXIS

- Implementar `Display` es implementar `fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result`. `write!(f, ...)` escribe en el formateador (como `format!`, pero hacia un destino).
- `Formatter<'_>`: el `'_` es un lifetime anónimo, "inferilo vos".
- `type Nombre = Tipo;` crea un **alias**, no un tipo nuevo.
- `Hash` y `Eq` permiten usar el tipo como clave de un `HashMap`. `Copy` tiene sentido para newtypes de números.

Ventajas: costo cero en runtime, el wrapper y el tipo envuelto no se confunden, y con el campo privado nadie accede al interior sin pasar por tu API.

Desventaja (según el libro): mucho boilerplate. Cada método o trait del tipo interno que quieras exponer necesita un "pasamanos". El crate `derive_more` genera muchos de esos derives (`From`, `Display`, `Add`, etc.).

DESDE KOTLIN / JAVA

Es tu `@JvmInline value class Email(val value: String)`. La diferencia es que en Rust el campo es privado por defecto, así que el constructor que valida es la **única** puerta de entrada (siempre que el struct esté en otro módulo).

3.2 Builder

Qué es: construir un objeto a través de llamadas encadenadas a un ayudante.

Por qué en Rust se usa mucho más que en Kotlin: el libro lo explica así: Rust **no tiene sobrecarga de funciones ni parámetros con valores por defecto**. En Kotlin resolvés todo con `Solicitud(url = "...", timeout = 5)`. En Rust, un struct con muchos campos opcionales pide un builder.

```rust
#[derive(Debug, Clone, Copy, PartialEq)]
enum Metodo { Get, Post }

#[derive(Debug)]
struct Solicitud {
    url: String,
    metodo: Metodo,
    headers: Vec<(String, String)>,
    timeout_seg: u64,
    cuerpo: Option<String>,
}

struct SolicitudBuilder {
    url: String,
    metodo: Metodo,
    headers: Vec<(String, String)>,
    timeout_seg: u64,
    cuerpo: Option<String>,
}

impl Solicitud {
    // Punto de entrada: hace que el builder sea fácil de descubrir
    fn builder(url: impl Into<String>) -> SolicitudBuilder {
        SolicitudBuilder {
            url: url.into(),
            metodo: Metodo::Get,
```

```
            headers: Vec::new(),
            timeout_seg: 30,
            cuerpo: None,
        }
    }
}

impl SolicitudBuilder {
    fn metodo(mut self, m: Metodo) -> Self {
        self.metodo = m;
        self
    }
    fn header(mut self, clave: &str, valor: &str) -> Self {
        self.headers.push((clave.into(), valor.into()));
        self
    }
    fn timeout(mut self, segundos: u64) -> Self {
        self.timeout_seg = segundos;
        self
    }
    fn cuerpo(mut self, c: impl Into<String>) -> Self {
        self.cuerpo = Some(c.into());
        self
    }
    // `build` puede validar combinaciones inválidas
    fn build(self) -> Result<Solicitud, String> {
        if self.metodo == Metodo::Get && self.cuerpo.is_some() {
            return Err("un GET no lleva cuerpo".into());
        }
        Ok(Solicitud {
            url: self.url,
            metodo: self.metodo,
            headers: self.headers,
            timeout_seg: self.timeout_seg,
            cuerpo: self.cuerpo,
        })
    }
}

fn main() {
    let ok = Solicitud::builder("https://api.ejemplo.com/eventos")
        .metodo(Metodo::Post)
        .header("Content-Type", "application/json")
        .cuerpo(r#"{"nombre":"Recital"}"#)
        .timeout(5)
        .build();
    println!("{ok:#?}");

    let mal = Solicitud::builder("https://x.com").cuerpo("hola").build();
    println!("{mal:?}");
}
```

SINTAXIS

- `fn metodo(mut self, ...) -> Self`: el builder se recibe **por valor**, se modifica y se devuelve. Eso permite encadenar `.a().b().c()`. El `mut` delante de `self` solo dice "dentro de esta función puedo modificar mi copia".
- `impl Into<String>` acepta cualquier cosa convertible a `String` (`&str`, `String`, etc.). Llamás `.into()` para convertir. Es una forma idiomática de hacer APIs cómodas.
- `r#"..."#` es un string crudo: las comillas internas no se escapan.

Variante por referencia: el libro menciona que a menudo es más ergonómico que los métodos reciban y devuelvan `&mut Self`. Eso permite tanto el estilo encadenado como el paso a paso (`let mut b = ...; b.a(); b.b(); b.build()`), por ejemplo cuando agregás headers dentro de un `if`. La librería estándar lo hace en `std::process::Command`.

Para no escribirlo a mano: el crate `derive_builder` genera el builder con un `#[derive(Builder)]`.

En nuestra arquitectura: sirve para objetos de configuración, consultas con muchos filtros opcionales (por ejemplo, una búsqueda de eventos por fecha, ciudad y precio) y datos de prueba en los tests. En la sección 3.9 vas a ver cómo hacer que el compilador exija los campos obligatorios.

3.3 Strategy: inversión de dependencias

Qué es: definir el esqueleto de un algoritmo en abstracto y separar las implementaciones concretas. El libro lo presenta explícitamente como la forma de aplicar el **principio de inversión de dependencias** (la D de SOLID): el código general depende de una abstracción (un trait), y las implementaciones concretas se adaptan a ella.

Los repositorios ya son el patrón Strategy: el caso de uso conoce el trait `UserRepository`, y la infraestructura elige la estrategia (Postgres, memoria).

Un ejemplo de negocio con tres formas de implementarlo:

```rust
// Los precios en centavos y en enteros: nunca uses f64 para plata
trait PoliticaDeDescuento {
    fn aplicar(&self, precio: u64) -> u64;
}

struct SinDescuento;
impl PoliticaDeDescuento for SinDescuento {
    fn aplicar(&self, precio: u64) -> u64 { precio }
}

struct Porcentaje(u8);
impl PoliticaDeDescuento for Porcentaje {
    fn aplicar(&self, precio: u64) -> u64 {
        precio - precio * self.0 as u64 / 100
    }
}

// Forma 1: genérico (estático). La estrategia se fija en compilación.
// `?Sized` permite que P también sea un `dyn PoliticaDeDescuento`.
fn total<P: PoliticaDeDescuento + ?Sized>(politica: &P, precios: &[u64]) -> u64 {
    precios.iter().map(|&p| politica.aplicar(p)).sum()
}

// Forma 2: dyn (dinámico). La estrategia se elige en runtime.
fn elegir_politica(codigo: &str) -> Box<dyn PoliticaDeDescuento> {
    match codigo {
        "PREVENTA" => Box::new(Porcentaje(20)),
        _ => Box::new(SinDescuento),
    }
}

// Forma 3: closure. Para estrategias chicas, no hace falta un trait.
fn total_con<F: Fn(u64) -> u64>(f: F, precios: &[u64]) -> u64 {
    precios.iter().map(|&p| f(p)).sum()
}

fn main() {
    let precios = [10_000, 25_000];
    println!("{}", total(&Porcentaje(10), &precios));

    let politica = elegir_politica("PREVENTA");
    println!("{}", total(politica.as_ref(), &precios));

    println!("{}", total_con(|p| p.saturating_sub(1_500), &precios));
}
```

SINTAXIS

- `self.0 as u64` convierte el `u8` a `u64`. `as` es el operador de conversión entre tipos numéricos.
- `10_000`: los guiones bajos en números son solo para leer mejor.
- `politica.as_ref()` convierte un `Box<dyn Trait>` en `&dyn Trait`.
- `P: Trait + ?Sized`: por defecto, todo parámetro genérico tiene que tener un **tamaño conocido en compilación** (`Sized`). Un `dyn Trait` no lo tiene, porque puede ser cualquier tipo. `?Sized` relaja esa exigencia. Sin él, `total(politica.as_ref(), ...)` no compila. Regla práctica: si una función genérica que recibe `&P` tiene que aceptar también objetos `dyn`, agregá `+ ?Sized`.
- `saturating_sub` resta sin pasar por debajo de cero (con `u64`, una resta normal que da negativo hace panic en modo debug).

Cuándo usar cada forma:

Forma	Cuándo
Genérico `<P: Trait>`	La estrategia se conoce en compilación; máximo rendimiento
`Box<dyn Trait>` / `Arc<dyn Trait>`	Se elige en runtime (configuración, inyección en `main`)
Closure `F: Fn(...)`	La estrategia es una función chica sin estado complejo

El libro menciona `serde` como ejemplo real de Strategy: el mismo struct se serializa a JSON o CBOR cambiando solo el crate de formato. Y `Option::map` es Strategy en miniatura: le pasás la estrategia como closure.

3.4 Command

Qué es: representar acciones como objetos para poder guardarlas, encolarlas, ejecutarlas más tarde o **deshacerlas**.

El ejemplo del libro son migraciones de base de datos que saben revertirse:

```rust
trait Migracion {
    fn ejecutar(&self) -> &str;
    fn revertir(&self) -> &str;
}

struct CrearTabla;
impl Migracion for CrearTabla {
    fn ejecutar(&self) -> &str { "create table" }
    fn revertir(&self) -> &str { "drop table" }
}

struct AgregarCampo;
impl Migracion for AgregarCampo {
    fn ejecutar(&self) -> &str { "add field" }
    fn revertir(&self) -> &str { "remove field" }
}

struct Esquema {
    comandos: Vec<Box<dyn Migracion>>,
}

impl Esquema {
    fn ejecutar(&self) -> Vec<&str> {
        self.comandos.iter().map(|c| c.ejecutar()).collect()
    }
    fn revertir(&self) -> Vec<&str> {
        self.comandos.iter().rev().map(|c| c.revertir()).collect() // en orden inverso
    }
}

// Variante con punteros a función: sin traits ni Box
type FnComando = fn() -> String;
struct Comando {
    ejecutar: FnComando,
    revertir: FnComando,
}

fn main() {
    let esquema = Esquema { comandos: vec![Box::new(CrearTabla), Box::new(AgregarCampo)] };
    assert_eq!(esquema.ejecutar(), vec!["create table", "add field"]);
    assert_eq!(esquema.revertir(), vec!["remove field", "drop table"]);

    let c = Comando {
        ejecutar: || "create index".to_string(),
        revertir: || "drop index".to_string(),
    };
    println!("{} / {}", (c.ejecutar)(), (c.revertir)());
}
```

SINTAXIS

- `fn() -> String` (con `f` minúscula) es el tipo **puntero a función**. Una closure que no captura nada se puede convertir en uno.
- `(c.ejecutar)()` lleva paréntesis porque `ejecutar` es un **campo** que contiene una función. Sin paréntesis, Rust buscaría un **método** llamado `ejecutar`.
- `.rev()` invierte el iterador.

El libro compara tres variantes: objetos trait (`Box<dyn Migracion>`), punteros a función y `Box<dyn Fn()>`. Su recomendación: si los comandos son chicos, las funciones evitan el despacho dinámico; si cada comando es un struct con estado y varias funciones, conviene el trait.

En nuestra arquitectura: los casos de uso con `execute` son más bien **command handlers** (reciben un comando con datos y lo ejecutan). El patrón Command completo tiene sentido cuando necesitás **guardar** las acciones: colas de trabajos, reintentos, auditoría o deshacer.

3.5 RAII con guardas

Qué es: adquirir un recurso al crear un objeto y liberarlo en su destructor (`Drop`). En Rust se lleva más lejos: el objeto funciona como **guarda**, y el sistema de tipos garantiza que solo accedés al recurso a través de ella y mientras exista.

El ejemplo clásico es `Mutex`: `lock()` devuelve un `MutexGuard`, y el candado se suelta solo cuando la guarda sale de alcance.

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let vendidas = Arc::new(Mutex::new(0u32));
    let mut hilos = Vec::new();

    for _ in 0..4 {
        let vendidas = Arc::clone(&vendidas);
        hilos.push(thread::spawn(move || {
            for _ in 0..1000 {
                let mut guarda = vendidas.lock().unwrap(); // toma el candado
                *guarda += 1;
            } // `guarda` sale de alcance: el candado se suelta solo
        }));
    }

    for h in hilos {
        h.join().unwrap();
    }
    println!("vendidas: {}", *vendidas.lock().unwrap());
}
```

No existe forma de olvidarse de soltar el candado, ni de tocar el contador sin haberlo tomado: el `u32` está **adentro** del `Mutex`.

Podés crear tus propias guardas. Por ejemplo, una transacción que hace **rollback automático** si nadie llamó a `commit`:

```rust
struct Transaccion<'a> {
    log: &'a mut Vec<String>,
    confirmada: bool,
}

impl<'a> Transaccion<'a> {
    fn iniciar(log: &'a mut Vec<String>) -> Self {
        log.push("BEGIN".into());
        Self { log, confirmada: false }
    }
    fn ejecutar(&mut self, sql: &str) {
        self.log.push(sql.to_string());
    }
    // Consume la transacción: después del commit no se puede seguir usando
    fn commit(mut self) {
        self.log.push("COMMIT".into());
        self.confirmada = true;
    } // acá corre `drop`, pero ya está confirmada
}

impl Drop for Transaccion<'_> {
    fn drop(&mut self) {
        if !self.confirmada {
            self.log.push("ROLLBACK".into());
        }
    }
}

fn reservar(log: &mut Vec<String>, falla: bool) -> Result<(), String> {
    let mut tx = Transaccion::iniciar(log);
    tx.ejecutar("UPDATE entradas SET estado = 'reservada'");
    if falla {
        return Err("sin stock".into()); // `tx` se destruye: ROLLBACK
    }
    tx.ejecutar("INSERT INTO reservas ...");
    tx.commit();
    Ok(())
}

fn main() {
    let mut log = Vec::new();
    let _ = reservar(&mut log, false);
    let _ = reservar(&mut log, true);
    println!("{log:#?}");
}
```

SINTAXIS

- `impl Drop for Transaccion<'_>`: el `'_` dice "para cualquier lifetime", sin necesidad de nombrarlo.
- `fn commit(mut self)`: recibe la transacción por valor. Después de `tx.commit()`, la variable `tx` está movida y el compilador no te deja volver a usarla. **No existe** el bug de "ejecutar después del commit".
- `let _ = reservar(...)`: descarta explícitamente el `Result` (si no, el compilador avisa que ignoraste un posible error).

Esta es exactamente la semántica de `sqlx::Transaction`: si se destruye sin `commit()`, hace rollback. Es el equivalente de `@Transactional` de Spring, pero sin proxies ni magia: lo garantiza el alcance de una variable.

3.6 Descomponé structs para préstamos independientes

Este patrón existe solo porque Rust tiene borrow checker. El problema típico: un método con `&mut self` toma prestado **todo** el struct, aunque solo use un campo.

Supongamos este código:

```
struct Evento {
    precios: Vec<u64>,
    vendidas: u32,
    recaudado: u64,
}

impl Evento {
    fn registrar(&mut self, precio: u64) {
        self.vendidas += 1;
        self.recaudado += precio;
    }

    fn procesar(&mut self) {
        for p in &self.precios {
            self.registrar(*p); // ERROR
        }
    }
}
```

El compilador lo rechaza:

```
error[E0502]: cannot borrow `*self` as mutable because it is also borrowed as immutable
   |
   |         for p in &self.precios {
   |                  -------------
   |                  |
   |                  immutable borrow occurs here
   |                  immutable borrow later used here
   |             self.registrar(*p);
   |             ^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
```

`registrar` pide `&mut self` completo mientras estamos leyendo `self.precios`. La solución que propone el libro es **separar** los datos que cambian juntos en su propio struct:

```rust
#[derive(Debug, Default)]
struct Estadisticas {
    vendidas: u32,
    recaudado: u64,
}

impl Estadisticas {
    fn registrar(&mut self, precio: u64) {
        self.vendidas += 1;
        self.recaudado += precio;
    }
}

struct Evento {
    precios: Vec<u64>,
    stats: Estadisticas,
}

impl Evento {
    fn procesar(&mut self) {
        for p in &self.precios {
            self.stats.registrar(*p); // campos distintos: préstamos independientes
        }
    }
}

fn main() {
    let mut e = Evento { precios: vec![1000, 2500], stats: Estadisticas::default() };
    e.procesar();
    println!("{:?}", e.stats);
}
```

El borrow checker sabe que `self.precios` y `self.stats` son campos **distintos** y permite prestarlos por separado. Como dice el libro, muchas veces este cambio revela una mejor abstracción: `Estadisticas` tenía sentido como concepto propio. Si un struct se resiste al borrow checker, suele ser señal de que está haciendo demasiadas cosas.

3.7 Preferí crates chicos

El libro recomienda crates pequeños que hagan una sola cosa bien. Ventajas: son más fáciles de entender, se reutilizan entre proyectos, y como **la unidad de compilación de Rust es el crate**, dividir un proyecto permite compilar partes en paralelo y recompilar solo lo que cambió.

Esto respalda directamente tu workspace con tres crates (`domain`, `application`, `infrastructure`). Además del orden arquitectónico, ganás tiempos de compilación.

Las desventajas que menciona:

- **Infierno de dependencias:** dos versiones del mismo crate generan tipos distintos e incompatibles. `cargo tree --duplicates` muestra qué dependencias aparecen repetidas en varias versiones.
- **crates.io no está curado:** un crate puede estar mal mantenido o ser malicioso. Revisá descargas, mantenimiento y quién lo publica antes de sumarlo.

3.8 Traits propios para simplificar bounds

Cuando una restricción genérica se vuelve ilegible (sobre todo con closures que devuelven `Result`), el libro sugiere **crear un trait con nombre** e implementarlo automáticamente para todos los tipos que cumplan el bound original. Esto se llama *blanket implementation*.

```rust
// Sin el patrón, cada struct y función repetiría: F: Fn(u64) -> Result<u64, String>
trait Tarifa {
    fn calcular(&self, base: u64) -> Result<u64, String>;
}

// Blanket impl: TODA closure con esa forma ya es una Tarifa
impl<F> Tarifa for F
where
    F: Fn(u64) -> Result<u64, String>,
{
    fn calcular(&self, base: u64) -> Result<u64, String> {
        self(base)
    }
}

// Y también un struct con nombre puede serlo
struct TarifaFija(u64);
impl Tarifa for TarifaFija {
    fn calcular(&self, base: u64) -> Result<u64, String> {
        Ok(base + self.0)
    }
}

fn cobrar<T: Tarifa>(tarifa: &T, base: u64) -> Result<u64, String> {
    tarifa.calcular(base)
}

fn main() {
    let con_limite = |base: u64| {
        if base > 100_000 { Err("excede el límite".to_string()) } else { Ok(base * 110 / 100) }
    };
    println!("{:?}", cobrar(&con_limite, 50_000));
    println!("{:?}", cobrar(&con_limite, 200_000));
    println!("{:?}", cobrar(&TarifaFija(500), 1_000));
}
```

SINTAXIS

- `impl<F> Tarifa for F where F: ...` se lee: "para todo tipo `F` que cumpla la condición, implemento `Tarifa`".
- `self(base)` invoca a la closure, porque en esa implementación `self` **es** la closure.

Ventaja: la firma `fn cobrar<T: Tarifa>` dice qué es la cosa, no cómo está hecha. **Desventaja:** quien lee tiene que descubrir que una closure común sirve como `Tarifa`, así que conviene documentarlo.

3.9 Genéricos como type classes: dividir la API por tipo

Este capítulo del libro conecta con el **typestate**: un tipo por estado, para que una operación en el estado incorrecto no compile. La idea: en Rust, `Vec<i32>` y `Vec<char>` son **tipos distintos**, y cada uno puede tener su propio bloque `impl`. Eso permite que ciertos métodos existan **solo** para ciertos parámetros genéricos.

El ejemplo del libro es una solicitud de archivo que puede venir por dos protocolos, donde solo uno tiene "punto de montaje". En lugar de un `Option` que todos tienen que chequear, el protocolo pasa a ser un parámetro del tipo. Aplicado a tu dominio:

```rust
// Estados como tipos
struct Disponible;
struct Reservada { usuario: u32 }
struct Pagada { usuario: u32, pago_id: String }

struct Entrada<E> {
    codigo: String,
    estado: E,
}

// Métodos para TODOS los estados
impl<E> Entrada<E> {
    fn codigo(&self) -> &str { &self.codigo }
}

// Métodos que existen SOLO en un estado
impl Entrada<Disponible> {
    fn nueva(codigo: &str) -> Self {
        Entrada { codigo: codigo.into(), estado: Disponible }
    }
    fn reservar(self, usuario: u32) -> Entrada<Reservada> {
        Entrada { codigo: self.codigo, estado: Reservada { usuario } }
    }
}

impl Entrada<Reservada> {
    fn usuario(&self) -> u32 { self.estado.usuario }
    fn pagar(self, pago_id: &str) -> Entrada<Pagada> {
        let usuario = self.estado.usuario;
        Entrada { codigo: self.codigo, estado: Pagada { usuario, pago_id: pago_id.into() } }
    }
}

impl Entrada<Pagada> {
    fn comprobante(&self) -> String {
        format!("{} pagada por {} ({})", self.codigo, self.estado.usuario, self.estado.pago_id)
    }
}

fn main() {
    let e = Entrada::nueva("A-0001");
    let e = e.reservar(7);
    println!("{} reservada por {}", e.codigo(), e.usuario());
    let e = e.pagar("pago-123");
    println!("{}", e.comprobante());
    // Entrada::nueva("B-1").pagar("x"); // no compila: `pagar` no existe para Entrada<Disponible>
}
```

SINTAXIS

- `struct Entrada<E>` declara un genérico sin restricciones.
- `impl<E> Entrada<E>` : "para cualquier `E` ". Estos métodos están disponibles en todos los estados.
- `impl Entrada<Reservada>` (sin `<E>` después de `impl`): implementación para un tipo **concreto**. Esos métodos solo existen cuando `E` es `Reservada` .
- Las transiciones reciben `self` por valor: la entrada vieja deja de existir y obtenés una con otro tipo.

Enum o typestate, la decisión práctica:

Situación	Elegí
El estado viene de la base de datos o de la red (se conoce en runtime)	**Enum** + `match`
Necesitás guardar entidades en distintos estados en una misma lista	**Enum**
Un flujo de pasos fijos dentro de una operación (builder, checkout, protocolo)	**Typestate**
Querés que llamar a una operación en el estado incorrecto sea imposible	**Typestate**

En una app backend típica, las entidades que cargás desde la base suelen ser enums (no sabés de antemano en qué estado están), y el typestate aparece en builders y flujos internos.

3.10 Los otros patrones del libro

Para completar el mapa, estos patrones también están en el libro, aunque son menos frecuentes en un backend de negocio:

- **Interpreter:** representar un lenguaje chico (expresiones, reglas) como un árbol de enums y evaluarlo. Útil para motores de reglas.
- **Visitor:** recorrer una estructura heterogénea (por ejemplo, el árbol del Interpreter) aplicando distintos algoritmos sin modificar los datos.
- **Fold:** como Visitor, pero produce una **nueva** versión de la estructura recorrida.
- **Contener `unsafe` en módulos chicos:** si necesitás código `unsafe`, aislalo en un módulo mínimo con una API segura encima.
- **FFI:** patrones para exponer o consumir código C. No aplica a tu caso por ahora.

Parte 4 · Antipatrones

4.1 Clonar para conformar al borrow checker

Qué es: cuando el borrow checker se queja, agregar `.clone()` hasta que compile.

```rust
fn main() {
    let mut x = 5;
    let y = &mut (x.clone()); // clonamos para "esquivar" el préstamo
    println!("{x}");          // compila...
    *y += 1;                  // ...pero `y` modifica una COPIA
    println!("{x}");          // `x` sigue valiendo 5
}
```

El problema no es solo el rendimiento: el clon es **otro dato**. Los cambios en uno no se reflejan en el otro, y si tu intención era modificar el original, introdujiste un bug que compila.

Clones que SÍ están bien:

- **`Arc::clone` y `Rc::clone`:** no copian el dato, solo incrementan un contador. En el `main` del proyecto de inducción, `Arc::clone(&register_user)` comparte el caso de uso entre REST y gRPC sin copiarlo. Escribirlo así (y no `register_user.clone()`) deja claro que es un `Arc`.
- **Cuando necesitás realmente una copia independiente:** por ejemplo, el repositorio en memoria devolvía `u.clone()` porque tiene que entregar un `User` propio sin perder el suyo.
- **Cuando la claridad vale más que el rendimiento:** el libro dice explícitamente que está bien en prototipos, cuando todavía estás aprendiendo ownership, o cuando la alternativa es mucho más complicada de leer.

Alternativas antes de clonar:

1. ¿Alcanza con prestar (`&`)? Muchas veces el clon sobra.
2. ¿El struct se resiste? Probá descomponerlo (sección 3.6).
3. ¿Estás cambiando de variante? `mem::take` (sección 2.3).
4. ¿Varios dueños de verdad? `Rc` o `Arc`.
5. Corré `cargo clippy`: detecta algunos clones innecesarios.

4.2 Polimorfismo con `Deref`: la tentación de la herencia

Este es el antipatrón más tentador para quien viene de Kotlin o Java. Imaginá que querés una base común para tus casos de uso:

```kotlin
abstract class BaseUseCase {
    fun log(msg: String) = println("[${javaClass.simpleName}] $msg")
}
class RegisterUserUseCase : BaseUseCase() { /* usa log() */ }
```

En Rust no hay herencia, y alguien descubre que implementando `Deref` puede "heredar" los métodos de otro struct:

```rust
use std::ops::Deref;

struct Base;
impl Base {
    fn log(&self, msg: &str) { println!("[base] {msg}"); }
}

struct RegistrarUsuario {
    base: Base,
}

// ANTIPATRÓN: usar Deref para simular `extends Base`
impl Deref for RegistrarUsuario {
    type Target = Base;
    fn deref(&self) -> &Base { &self.base }
}

fn main() {
    let caso = RegistrarUsuario { base: Base };
    caso.log("funciona... pero no lo hagas"); // el punto busca métodos en Base vía Deref
}
```

Por qué está mal, según el libro:

- **Sorprende:** `Deref` existe para punteros inteligentes (`Box`, `Arc`, `String` → `str`), no para convertir entre tipos sin relación. Quien lea el código no va a esperar esto.
- **No crea subtipos:** `RegistrarUsuario` **no** es un `Base`. Los traits que implementa `Base` no se heredan, así que no podés pasar un `RegistrarUsuario` donde se espera un `T: AlgunTrait` de `Base`.
- **`self` cambia de significado:** dentro de `log`, `self` es el `Base`, no el caso de uso. No hay "métodos virtuales".
- **Solo herencia simple**, sin interfaces ni privacidad por clase.

Qué hacer en su lugar:

```rust
// Opción 1: trait con método por defecto (comportamiento compartido)
trait Loggeable {
    fn nombre(&self) -> &'static str;
    fn log(&self, msg: &str) {
        println!("[{}] {msg}", self.nombre());
    }
}

struct RegistrarUsuario;
impl Loggeable for RegistrarUsuario {
    fn nombre(&self) -> &'static str { "RegistrarUsuario" }
}

// Opción 2: composición con delegación explícita
struct Logger { prefijo: String }
impl Logger {
    fn log(&self, msg: &str) { println!("[{}] {msg}", self.prefijo); }
}

struct PagarEntrada { logger: Logger }
impl PagarEntrada {
    fn execute(&self) {
        self.logger.log("pagando"); // se ve de dónde viene
    }
}

fn main() {
    RegistrarUsuario.log("registrando");
    PagarEntrada { logger: Logger { prefijo: "PagarEntrada".into() } }.execute();
}
```

Si la delegación manual se vuelve tediosa, el libro menciona los crates `delegate` y `ambassador`, que la generan.

Y en la práctica: para cosas transversales como el logging, en Rust se usa el crate `tracing` (con `#[tracing::instrument]` sobre la función), así que muchas veces la "clase base" directamente no hace falta.

4.3 `#![deny(warnings)]`

Qué es: poner `#![deny(warnings)]` en la raíz del crate para que cualquier warning rompa la compilación.

Por qué es un problema: Rust introduce lints nuevos como *warnings* durante un período de gracia antes de convertirlos en errores, y depreca APIs con warnings. Con `deny(warnings)`, **actualizar el compilador puede romper tu build** sin que hayas cambiado una línea. Renunciás a la estabilidad que Rust promete.

Alternativas del libro:

1. Sacar la configuración del código y ponerla en el CI:

```
RUSTFLAGS="-D warnings" cargo build
```

2. Denegar **lints específicos** con nombre, en lugar de "todos los warnings".

Hoy, además, Cargo permite configurar lints de forma centralizada en el `Cargo.toml` (desde Rust 1.74), ideal para un workspace:

```toml
# Cargo.toml del workspace
[workspace.lints.rust]
unsafe_code = "forbid"

[workspace.lints.clippy]
unwrap_used = "warn"

# Cargo.toml de cada crate miembro
[lints]
workspace = true
```

Con `unwrap_used`, clippy te avisa cada vez que usás `.unwrap()` en vez de manejar el error, algo útil en las capas de dominio y aplicación.

Parte 5 · Nuestra arquitectura

5.1 Las capas y quién depende de quién

Todos nuestros servicios son un **workspace de Cargo** (un repositorio con varios crates que se compilan juntos) con estas capas. Es Clean Architecture: el negocio en el centro y los detalles técnicos afuera.

```
induccion/
├── Cargo.toml                  workspace: miembros, versiones y lints compartidos
├── migrations/                 SQL del esquema
└── crates/
    ├── domain/                 entidades, value objects, errores de dominio. Sin I/O.
    ├── application/            casos de uso, puertos (traits), reglas de negocio
    ├── proto/                  contrato gRPC (.proto) y código generado. Solo si hay gRPC.
    └── infrastructure/         axum, tonic, tokio-postgres, main
```

Crate	Contiene	Puede depender de
`domain`	Entidades, value objects (newtypes), estados como `enum`, errores de dominio. Lógica pura.	`std` y crates chicos y puros (`thiserror`, `uuid`).
`application`	Un struct por caso de uso con un `execute`; puertos (traits de repositorio con `Send + Sync`); comandos; reglas de `validations_rs`; errores de aplicación.	`domain`, `validations_rs`, `thiserror`. Nada de transporte, base de datos ni UI.
`proto`	Solo con gRPC: los `.proto`, un `build.rs` que genera el código y el código generado.	`tonic`, `prost`, `tonic-prost`.
`infrastructure`	Adaptadores (handlers de axum, servicios de tonic, repositorios de PostgreSQL), mapeo de errores a HTTP o gRPC, configuración y `main`.	Todo lo anterior y el stack.
Apps Tauri o Dioxus	Un crate por app. Son adaptadores, como `infrastructure`.	`application` si corren en el mismo proceso; un cliente de la API si no.

Tres reglas que el compilador y la revisión de EGM hacen cumplir:

1. **Las dependencias apuntan hacia adentro**: `infrastructure` → `application` → `domain`. Como cada capa es un crate, Cargo impide importar algo que no está declarado como dependencia: `domain` no puede usar axum aunque quiera.
2. **Los tipos de transporte y de base de datos no cruzan hacia adentro.** Un mensaje generado por `prost`, un extractor de axum o una `tokio_postgres::Row` se traducen a tipos del dominio en el adaptador.
3. **Los adaptadores solo traducen.** Un handler, un servicio gRPC, un comando de Tauri o una server function de Dioxus convierten la entrada en un comando, llaman a un caso de uso y convierten el resultado. Sin reglas de negocio.

5.2 Una responsabilidad por tipo

Es una preferencia explícita del equipo, y el revisor de EGM la marca como error importante:

- **Un caso de uso es un struct con un solo método público, `execute`.** `RegisterUserUseCase`, no un `UserService` con diez métodos.
- **Una regla de negocio es un struct** que implementa `BusinessLogicValidation` o `AsyncBusinessLogicValidation` de `validations_rs`.
- **Un trait de repositorio por agregado.**
- **Un archivo por cada tipo público**, agrupados en módulos por rol (`use_cases/`, `validations/`, `ports/`), como los paquetes de Kotlin o Java.
- Si un caso de uso crece una segunda responsabilidad, se divide en dos, y un tercer caso de uso (o quien llama) los compone.

Así quedan los archivos de `application` en el proyecto de inducción:

```
application/src/
├── commands/register_user_command.rs      la entrada cruda del caso de uso
├── errors/register_user_error.rs          todo lo que puede salir mal
├── ports/user_repository.rs               el trait que implementa infrastructure
├── ports/repository_error.rs
├── use_cases/register_user.rs             RegisterUserUseCase::execute
└── validations/
    ├── adult_only.rs                      regla sincrónica
    ├── terms_accepted.rs                  regla sincrónica
    └── email_not_taken.rs                 regla asincrónica (consulta el repositorio)
```

5.3 De Kotlin y Spring a nuestro stack

Pieza en Kotlin / Spring	En nuestros proyectos	Dónde se explica
Módulos de Gradle por capa	Crates de un workspace de Cargo	6.1
`data class` inmutable	`struct` con `#[derive(...)]`	1.4
Value object	Newtype con campo privado y `TryFrom`	3.1 y 5.4
`sealed class`	`enum`	1.5
Excepciones	`enum` de error + `Result`, con `thiserror`	6.2
Interfaces de repositorio	`trait` con `Send + Sync`	5.4
`UseCase` con `execute`	Struct con `execute`, uno por acción	5.4
Clases de validación	Reglas de `validations_rs`, una por struct	6.4
Coroutines	`async` / `await` sobre Tokio	6.3
JPA / Spring Data	`tokio-postgres` con un pool `deadpool-postgres`	6.5
`@Transactional`	`Transaction` de tokio-postgres (rollback al destruirse)	3.5 y 6.5
`@RestController` y `@ControllerAdvice`	Handlers de axum y un error que implementa `IntoResponse`	6.6
gRPC con grpc-java	`tonic` + crate `proto`	6.7
Inyección de dependencias de Spring	Armado manual en `main` con `Arc<dyn Trait>`	5.4

Pieza en Kotlin / Spring	En nuestros proyectos	Dónde se explica
Android / desktop	Tauri 2 o Dioxus 0.7	6.8 y 6.9

5.4 El núcleo, archivo por archivo

Este es el código real de `domain` y `application` del proyecto de inducción. Compila, y sus tests pasan con `cargo test`.

Value object con validación de formato (`domain`). Tener un `Email` prueba que el formato se validó, porque `TryFrom` es la única forma de construirlo:

▸ crates/domain/src/value_objects/email.rs

```rust
use crate::errors::DomainError;

/// An email address that passed the format check. The only way to build one
/// is `TryFrom<&str>`, so holding an `Email` proves the check ran.
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct Email(String);

impl Email {
    pub fn as_str(&self) -> &str {
        &self.0
    }
}

impl TryFrom<&str> for Email {
    type Error = DomainError;

    fn try_from(raw: &str) -> Result<Self, Self::Error> {
        let normalized = raw.trim().to_lowercase();
        match normalized.split_once('@') {
            Some((local, domain)) if !local.is_empty() && domain.contains('.') =>
Ok(Email(normalized)),
            _ => Err(DomainError::InvalidEmail(raw.to_string())),
        }
    }
}
```

El puerto (`application`). El trait que la infraestructura implementa. Devuelve futures en caja (`RepoFuture`) para poder usarse como `Arc<dyn UserRepository>`:

▸ crates/application/src/ports/user_repository.rs

```rust
use std::future::Future;
use std::pin::Pin;

use domain::entities::User;
use domain::value_objects::{Email, UserId};

use super::RepositoryError;

/// Boxed future so the trait stays dyn-compatible (`Arc<dyn UserRepository>`).
pub type RepoFuture<'a, T> = Pin<Box<dyn Future<Output = Result<T, RepositoryError>> + Send + 'a>>;

/// Port for the user aggregate. Implemented in `infrastructure` with tokio-postgres.
pub trait UserRepository: Send + Sync {
    fn exists_by_email<'a>(&'a self, email: &'a Email) -> RepoFuture<'a, bool>;
    fn next_id(&self) -> RepoFuture<'_, UserId>;
    fn save<'a>(&'a self, user: &'a User) -> RepoFuture<'a, ()>;
}
```

SINTAXIS

- `Pin<Box<dyn Future<Output = ...> + Send + 'a>>` es "un future cualquiera, guardado en el heap, que se puede mandar a otro hilo y que vive lo que vive `'a`". Hace falta porque un trait con métodos `async fn` todavía no se puede usar como `dyn`. Las implementaciones devuelven `Box::pin(async move { ... })`.
- `RepoFuture<'_, UserId>`: el `'_` le pide al compilador que use el lifetime de `&self`.

Una regla de negocio sincrónica y una asincrónica (`application`, con `validations_rs`, ver 6.4):

▸ crates/application/src/validations/adult_only.rs

```rust
use validations_rs::{BusinessLogicValidation, BusinessRuleError, MinValidation};

use crate::commands::RegisterUserCommand;

/// Business rule: only adults can register.
pub struct AdultOnly;

impl BusinessLogicValidation<RegisterUserCommand> for AdultOnly {
    fn validate(&self, command: &RegisterUserCommand) -> Result<(), BusinessRuleError> {
        MinValidation::new(18u8).with_message("you must be of legal age").validate(&command.age)
    }
}
```

▸ crates/application/src/validations/email_not_taken.rs

```rust
use std::sync::Arc;

use domain::value_objects::Email;
use validations_rs::{
    AsyncBusinessLogicValidation, AsyncValidationError, BusinessRuleError, RuleCode, ValidationFuture,
};

use crate::ports::UserRepository;

/// Business rule that needs I/O: the email is not registered yet.
pub struct EmailNotTaken {
    repo: Arc<dyn UserRepository>,
}

impl EmailNotTaken {
    pub const CODE: RuleCode = RuleCode("EMAIL_TAKEN");

    pub fn new(repo: Arc<dyn UserRepository>) -> Self {
        Self { repo }
    }
}

impl AsyncBusinessLogicValidation<Email> for EmailNotTaken {
    fn validate<'a>(&'a self, email: &'a Email) -> ValidationFuture<'a> {
        Box::pin(async move {
            // A storage failure is not a broken rule: report it as `CheckFailed`.
            let taken =
self.repo.exists_by_email(email).await.map_err(AsyncValidationError::check_failed)?;
            if taken {
                Err(BusinessRuleError::new(Self::CODE, "the email is already registered")
                    .with_detail("email", email.as_str())
                    .into())
            } else {
                Ok(())
            }
        })
    }
}
```

El error del caso de uso. Cada variante es algo que el adaptador traduce de forma distinta (422, 503, 500):

▸ crates/application/src/errors/register_user_error.rs

```rust
use domain::errors::DomainError;
use validations_rs::{AsyncValidationError, BusinessRuleError};

use crate::ports::RepositoryError;

/// Everything that can go wrong when registering a user.
#[derive(Debug, thiserror::Error)]
pub enum RegisterUserError {
    /// The input breaks a format invariant (domain).
    #[error(transparent)]
    InvalidInput(#[from] DomainError),
    /// The input breaks a business rule (application).
    #[error(transparent)]
    BusinessRule(#[from] BusinessRuleError),
    /// Storage failed while saving.
    #[error(transparent)]
    Repository(#[from] RepositoryError),
    /// A rule could not be checked (for example, the database did not answer).
    #[error("a business rule could not be checked")]
    RuleCheckFailed(#[source] AsyncValidationError),
}

/// A violated async rule is a business error; a failed check is not.
impl From<AsyncValidationError> for RegisterUserError {
    fn from(error: AsyncValidationError) -> Self {
        match error {
            AsyncValidationError::Violation(rule) => RegisterUserError::BusinessRule(rule),
            other => RegisterUserError::RuleCheckFailed(other),
        }
    }
}
```

El caso de uso. Primero el formato (dominio), después las reglas baratas y por último las que hacen I/O:

▸ crates/application/src/use_cases/register_user.rs

```rust
use std::sync::Arc;

use domain::entities::User;
use domain::value_objects::Email;
use validations_rs::{start_validations, start_validations_async};

use crate::commands::RegisterUserCommand;
use crate::errors::RegisterUserError;
use crate::ports::UserRepository;
use crate::validations::{AdultOnly, EmailNotTaken, TermsAccepted};

/// One use case, one responsibility: register a user.
pub struct RegisterUserUseCase {
    repo: Arc<dyn UserRepository>,
    email_not_taken: EmailNotTaken,
}

impl RegisterUserUseCase {
    pub fn new(repo: Arc<dyn UserRepository>) -> Self {
        Self { email_not_taken: EmailNotTaken::new(Arc::clone(&repo)), repo }
    }

    pub async fn execute(&self, command: RegisterUserCommand) -> Result<User, RegisterUserError> {
        // 1. Format invariant: owned by the domain newtype.
        let email = Email::try_from(command.email.as_str())?;
        // 2. Cheap business rules first, in order.
        start_validations(&[&AdultOnly, &TermsAccepted], &command)?;
        // 3. Rules that need I/O, only if the cheap ones passed.
        start_validations_async(&[&self.email_not_taken], &email).await?;

        let user = User { id: self.repo.next_id().await?, email };
        self.repo.save(&user).await?;
        Ok(user)
    }
}
```

Los tests usan una implementación en memoria del puerto: el "mock" es otra implementación del trait.

▸ crates/application/tests/register_user.rs

```rust
use std::future::Future;
use std::pin::pin;
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Waker};

use application::commands::RegisterUserCommand;
use application::errors::RegisterUserError;
use application::ports::{RepoFuture, UserRepository};
use application::use_cases::RegisterUserUseCase;
use application::validations::EmailNotTaken;
use domain::entities::User;
use domain::errors::DomainError;
use domain::value_objects::{Email, UserId};
use validations_rs::MinValidation;

/// Test double: the port implemented in memory.
#[derive(Default)]
struct InMemoryUsers {
    users: Mutex<Vec<User>>,
    last_id: AtomicU64,
}

impl UserRepository for InMemoryUsers {
```

```rust
    fn exists_by_email<'a>(&'a self, email: &'a Email) -> RepoFuture<'a, bool> {
        Box::pin(async move { Ok(self.users.lock().expect("lock").iter().any(|u| &u.email == email)) })
    }
    fn next_id(&self) -> RepoFuture<'_, UserId> {
        Box::pin(async { Ok(UserId(self.last_id.fetch_add(1, Ordering::SeqCst) + 1)) })
    }
    fn save<'a>(&'a self, user: &'a User) -> RepoFuture<'a, ()> {
        Box::pin(async move {
            self.users.lock().expect("lock").push(user.clone());
            Ok(())
        })
    }
}

/// Minimal executor for futures that never pend (no runtime needed in tests).
fn block_on<F: Future>(future: F) -> F::Output {
    let mut future = pin!(future);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        if let Poll::Ready(out) = future.as_mut().poll(&mut cx) {
            return out;
        }
    }
}

fn command(email: &str, age: u8, accepted_terms: bool) -> RegisterUserCommand {
    RegisterUserCommand { email: email.into(), age, accepted_terms }
}

#[test]
fn registers_then_rejects_duplicates_and_rule_violations() {
    let use_case = RegisterUserUseCase::new(Arc::new(InMemoryUsers::default()));

    let user = block_on(use_case.execute(command("Ana@Example.com", 30, true))).expect("valid");
    assert_eq!(user.email.as_str(), "ana@example.com");

    match block_on(use_case.execute(command("ana@example.com", 30, true))) {
        Err(RegisterUserError::BusinessRule(e)) => assert_eq!(e.code, EmailNotTaken::CODE),
        other => panic!("expected EMAIL_TAKEN, got {other:?}"),
    }
    match block_on(use_case.execute(command("teen@example.com", 16, true))) {
        Err(RegisterUserError::BusinessRule(e)) => assert_eq!(e.code, MinValidation::<u8>::CODE),
        other => panic!("expected VALUE_BELOW_MIN, got {other:?}"),
    }
    assert!(matches!(
        block_on(use_case.execute(command("no-arroba", 30, true))),
        Err(RegisterUserError::InvalidInput(DomainError::InvalidEmail(_)))
    ));
}

/// A repository whose queries fail, as if the database were down.
struct DownUsers;

impl UserRepository for DownUsers {
    fn exists_by_email<'a>(&'a self, _: &'a Email) -> RepoFuture<'a, bool> {
        Box::pin(async { Err(application::ports::RepositoryError("connection refused".into())) })
    }
    fn next_id(&self) -> RepoFuture<'_, UserId> {
        Box::pin(async { Ok(UserId(0)) })
    }
    fn save<'a>(&'a self, _: &'a User) -> RepoFuture<'a, ()> {
        Box::pin(async { Ok(()) })
    }
}

#[test]
fn a_database_failure_is_not_reported_as_a_broken_rule() {
```

```
    let use_case = RegisterUserUseCase::new(Arc::new(DownUsers));
    assert!(matches!(
        block_on(use_case.execute(command("ana@example.com", 30, true))),
        Err(RegisterUserError::RuleCheckFailed(_))
    ));
}

/// Web frameworks (axum, tonic) run handlers on a multi-threaded runtime, so the use-case
/// future must be `Send`. This test fails to compile if it ever stops being `Send`.
#[test]
fn the_use_case_future_is_send() {
    fn assert_send<T: Send>(_: &T) {}
    let use_case = RegisterUserUseCase::new(Arc::new(InMemoryUsers::default()));
    let future = use_case.execute(command("ana@example.com", 30, true));
    assert_send(&future);
}
```

5.5 SOLID en Rust

- **Responsabilidad única (S):** un struct por caso de uso y por regla (5.2). Una función libre también tiene una sola responsabilidad: usala para pasos sin estado ni dependencias.
- **Abierto/cerrado (O):** los **traits** permiten agregar implementaciones sin tocar el código existente. Los **enums** son lo opuesto, cerrados a propósito: agregar una variante obliga a revisar cada `match`, y el compilador te dice dónde.
- **Sustitución de Liskov (L):** no hay subtipos por herencia; el contrato es el trait. Documentá qué espera cada método (por ejemplo, "devuelve `Ok(false)` si no existe, nunca `Err`").
- **Segregación de interfaces (I):** traits chicos. Si un caso de uso solo lee, puede depender de un trait de solo lectura.
- **Inversión de dependencias (D):** es el patrón Strategy (3.3): `application` define el trait y `infrastructure` lo implementa.

Un principio menos conocido encaja con todo lo anterior, el de **elección única** (Bertrand Meyer): "cuando un sistema tiene que soportar un conjunto de alternativas, un solo módulo debería conocer la lista completa". Es un `enum` definido en un único lugar, con el compilador avisando de cada `match` desactualizado.

Parte 6 · Inducciones por tecnología

Cada módulo explica una tecnología a través de su lugar en el proyecto de inducción: qué es, por qué la usamos, el código real, las reglas del equipo y los errores comunes. Termina con un **checkpoint**: algo que tenés que poder hacer antes de seguir.

Versiones de referencia (noviembre de 2026): `tokio` 1.54, `axum` 0.8.9, `tokio-postgres` 0.7.18, `deadpool-postgres` 0.14.2, `tonic` 0.14.9, Tauri 2.12, Dioxus 0.7.10, `validations_rs` 0.2.0. Ante una duda de API, la referencia es `cargo doc --open` sobre tu versión exacta.

6.1 Cargo y workspaces

Qué es. Cargo compila, gestiona dependencias, corre tests y publica. Un **crate** es la unidad de compilación (una librería o un binario). Un **workspace** agrupa varios crates que comparten `Cargo.lock`, carpeta `target/` y configuración.

El `Cargo.toml` del workspace declara los miembros, las versiones de todas las dependencias (una sola vez) y los lints:

▸ Cargo.toml

```toml
[workspace]
members = ["crates/domain", "crates/application", "crates/proto", "crates/infrastructure"]
resolver = "2"

[workspace.package]
edition = "2021"
rust-version = "1.85"

[workspace.dependencies]
# Our crates
domain = { path = "crates/domain" }
application = { path = "crates/application" }
proto = { path = "crates/proto" }
# Business-rule validations (change to "0.2" once 0.2.0 is published on crates.io)
validations_rs = { git = "https://github.com/mdambro/validations_rs", branch = "release/0.2.0" }
# Stack
anyhow = "1"
axum = "0.8"
deadpool-postgres = "0.14"
prost = "0.14"
serde = { version = "1", features = ["derive"] }
thiserror = "2"
tokio = { version = "1", features = ["rt-multi-thread", "macros", "net"] }
tokio-postgres = "0.7"
tonic = "0.14"
tonic-prost = "0.14"
tonic-prost-build = "0.14"

[workspace.lints.rust]
unsafe_code = "forbid"

[workspace.lints.clippy]
unwrap_used = "warn"
```

El **`Cargo.toml` de un crate** toma las dependencias del workspace con `workspace = true`:

▸ crates/infrastructure/Cargo.toml

```toml
[package]
name = "infrastructure"
version = "0.1.0"
edition.workspace = true
rust-version.workspace = true

[dependencies]
domain.workspace = true
application.workspace = true
proto.workspace = true
anyhow.workspace = true
axum.workspace = true
deadpool-postgres.workspace = true
serde.workspace = true
tokio.workspace = true
tokio-postgres.workspace = true
tonic.workspace = true

[lints]
workspace = true
```

CONCEPTOS DE CARGO

- **Versiones**: `"0.8"` significa "cualquier 0.8.x compatible" (semver). `Cargo.lock` fija la versión exacta usada; se versiona en el repositorio para que todos compilen lo mismo.
- **Features**: interruptores de compilación de un crate. `tokio = { version = "1", features = ["rt-multi-thread", "macros", "net"] }` activa solo las partes de Tokio que usamos. Las features se suman: si dos crates piden features distintas del mismo crate, se activan todas.
- **`[build-dependencies]`**: dependencias que solo usa `build.rs`, un programa que corre antes de compilar el crate (en `proto`, genera el código gRPC).
- **`[workspace.lints]` + `[lints] workspace = true`**: los mismos lints para todos los crates, sin `#![deny(warnings)]` en el código (4.3).

Comandos útiles en un workspace: `cargo build -p application` (un solo crate), `cargo test -p domain`, `cargo tree -p infrastructure` (árbol de dependencias), `cargo tree --duplicates` (crates repetidos en varias versiones).

Checkpoint. Agregá `uuid = "1"` al workspace y usalo desde `domain` con `workspace = true`. Corré `cargo tree -p domain` y encontralo en el árbol. Después sacalo.

6.2 Errores con `thiserror` y `anyhow`

Qué son. En Rust los errores son valores (1.6). Dos crates reducen el código repetitivo:

- **`thiserror`** genera `Display`, `std::error::Error` y las conversiones `From` para tus enums de error. Se usa en `domain`, `application` y los adaptadores: los errores tienen que poder matchearse variante por variante.
- **`anyhow`** es un error opaco ("cualquier error, con contexto"). Se usa **solo en `main`** y en otros binarios de nivel superior, donde ya nadie va a matchear el error, solo informarlo.

El error del caso de uso del proyecto de inducción:

▸ crates/application/src/errors/register_user_error.rs

```rust
use domain::errors::DomainError;
use validations_rs::{AsyncValidationError, BusinessRuleError};

use crate::ports::RepositoryError;

/// Everything that can go wrong when registering a user.
#[derive(Debug, thiserror::Error)]
pub enum RegisterUserError {
    /// The input breaks a format invariant (domain).
    #[error(transparent)]
    InvalidInput(#[from] DomainError),
    /// The input breaks a business rule (application).
    #[error(transparent)]
    BusinessRule(#[from] BusinessRuleError),
    /// Storage failed while saving.
    #[error(transparent)]
    Repository(#[from] RepositoryError),
    /// A rule could not be checked (for example, the database did not answer).
    #[error("a business rule could not be checked")]
    RuleCheckFailed(#[source] AsyncValidationError),
}

/// A violated async rule is a business error; a failed check is not.
impl From<AsyncValidationError> for RegisterUserError {
    fn from(error: AsyncValidationError) -> Self {
        match error {
            AsyncValidationError::Violation(rule) => RegisterUserError::BusinessRule(rule),
            other => RegisterUserError::RuleCheckFailed(other),
        }
    }
}
```

SINTAXIS DE `THISERROR`

- `#[derive(Debug, thiserror::Error)]` sobre un enum o struct.
- `#[error("texto {0}")]` define el `Display`. `{0}` es el primer campo de una variante posicional; `{campo}` uno con nombre.
- `#[error(transparent)]` delega `Display` y `source` en el error interno.
- `#[from]` genera `impl From<Interno> for MiError`, lo que habilita `?`. `#[source]` marca la causa sin generar `From`.
- Cuando una conversión necesita lógica (como `AsyncValidationError`, que se reparte en dos variantes), se escribe el `From` a mano.

`anyhow` en `main`, con `.context(...)` para agregar información a un error sin definir tipos nuevos:

```rust
// fragmento de main.rs (crate infrastructure)
let database_url = std::env::var("DATABASE_URL").context("DATABASE_URL is not set")?;
```

Reglas del equipo. `thiserror` en todas las capas; `anyhow` solo en `main`. Los mensajes de error son para personas; las decisiones se toman por variante o por código. Un error que se muestra al cliente no expone detalles internos (SQL, rutas, trazas).

Errores comunes. Usar `anyhow::Result` en `application`: el adaptador ya no puede distinguir un 422 de un 500. Usar `.unwrap()` para "después lo arreglo": en producción es un panic.

Checkpoint. Agregá a `DomainError` una variante `EmptyName` con su mensaje, y escribí un test que compruebe su `to_string()`.

6.3 Async con Tokio

Qué es. Rust tiene `async` / `await` en el lenguaje, pero no trae un runtime que ejecute las tareas: lo elige cada aplicación. Usamos **Tokio**, el estándar del ecosistema, y es el que necesitan axum, tonic y tokio-postgres.

La diferencia clave con Kotlin o JavaScript: los futures son perezosos. Llamar a una función `async` no ejecuta nada; devuelve un *future*. El trabajo ocurre cuando alguien lo espera con `.await` o lo entrega al runtime con `tokio::spawn`.

```rust
use std::time::Duration;

// Necesita tokio con las features "rt-multi-thread", "macros" y "time".
#[tokio::main]
async fn main() {
    // Esperar: el future corre acá, y esta función se pausa hasta que termina.
    let total = sumar(2, 3).await;

    // Tarea en paralelo: corre en otro hilo del runtime; el JoinHandle es su resultado.
    let tarea = tokio::spawn(async move { sumar(total, 10).await });

    // Código bloqueante (I/O síncrona, CPU): a un hilo aparte, nunca en una tarea async.
    let hash = tokio::task::spawn_blocking(|| calcular_hash("dato")).await.expect("la tarea no falló");

    // Esperar varias cosas a la vez y fallar si una falla.
    let (a, b) = tokio::join!(sumar(1, 1), sumar(2, 2));

    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{} {hash} {a} {b}", tarea.await.expect("la tarea no falló"));
}

async fn sumar(a: u64, b: u64) -> u64 {
    a + b
}

fn calcular_hash(dato: &str) -> u64 {
    dato.bytes().map(u64::from).sum()
}
```

SINTAXIS

- `async fn` devuelve un `impl Future<Output = T>`. `.await` solo se puede usar dentro de algo `async`.
- `#[tokio::main]` convierte `async fn main` en un `main` normal que arranca el runtime.
- `async move { ... }` es un bloque async que se adueña de lo que captura, como una closure `move` (1.10).
- `tokio::spawn` exige que el future sea `Send + 'static`: que se pueda mandar a otro hilo y que no tome prestado nada local.

`Send`: el error de compilación que más vas a ver. Si una tarea guarda algo que no es `Send` (por ejemplo un `Rc`, o un guard de `std::sync::Mutex`) a través de un `.await`, no compila. Por eso el proyecto de inducción tiene un test que falla si el future del caso de uso deja de ser `Send`:

```rust
// tests/register_user.rs (crate application)
#[test]
fn the_use_case_future_is_send() {
    fn assert_send<T: Send>(_: &T) {}
    let use_case = RegisterUserUseCase::new(Arc::new(InMemoryUsers::default()));
    let future = use_case.execute(command("ana@example.com", 30, true));
    assert_send(&future);
}
```

Reglas del equipo.

- Los binarios activan `rt-multi-thread` y `macros`; las librerías, solo las features que usan.
- Nada bloqueante dentro de una tarea async: I/O síncrona y trabajo pesado de CPU van a `spawn_blocking` (o a un pool como `rayon`).
- Para estado compartido, `std::sync::Mutex` dentro de un `Arc`. `tokio::sync::Mutex` solo si el guard tiene que sobrevivir a un `.await` (la documentación de Tokio y la de Tauri dan la misma recomendación).

Errores comunes. Olvidar el `.await` (el compilador avisa: "futures do nothing unless you `.await` or poll them"). Llamar a una API bloqueante (`std::fs`, `std::thread::sleep`) dentro de un handler: frena a todas las tareas de ese hilo.

Checkpoint. Escribí un programa que lance 3 tareas con `tokio::spawn` que duerman 100, 200 y 300 ms, y que espere las tres con `JoinHandle`. Medí con `std::time::Instant` que el total ronda los 300 ms y no los 600.

6.4 Reglas de negocio con `validations_rs` 0.2

Qué es. Nuestra librería de validaciones para la capa `application`. Cada regla de negocio es un struct (una responsabilidad) que implementa un trait; un caso de uso las corre en orden, al principio de `execute`.

Qué va en `validations_rs` y qué no:

Tipo de validación	Dónde va	Ejemplo
Formato de un valor suelto	Constructor del newtype en `domain` (`TryFrom`)	El email tiene `@` y dominio.
Regla de negocio sobre la entrada	Regla sincrónica, `BusinessLogicValidation`	Mayor de edad, términos aceptados.
Regla que necesita I/O	Regla asincrónica, `AsyncBusinessLogicValidation`	El email no está registrado.

Una regla sincrónica delega en una del catálogo (`MinValidation`) y le pone un mensaje propio; el código de error (`VALUE_BELOW_MIN`) no cambia:

▸ crates/application/src/validations/adult_only.rs

```rust
use validations_rs::{BusinessLogicValidation, BusinessRuleError, MinValidation};

use crate::commands::RegisterUserCommand;

/// Business rule: only adults can register.
pub struct AdultOnly;

impl BusinessLogicValidation<RegisterUserCommand> for AdultOnly {
    fn validate(&self, command: &RegisterUserCommand) -> Result<(), BusinessRuleError> {
        MinValidation::new(18u8).with_message("you must be of legal age").validate(&command.age)
    }
}
```

Una regla asincrónica distingue "no cumple" de "no se pudo verificar", la gran novedad de la versión 0.2:

▸ crates/application/src/validations/email_not_taken.rs

```rust
use std::sync::Arc;

use domain::value_objects::Email;
use validations_rs::{
    AsyncBusinessLogicValidation, AsyncValidationError, BusinessRuleError, RuleCode, ValidationFuture,
};

use crate::ports::UserRepository;

/// Business rule that needs I/O: the email is not registered yet.
pub struct EmailNotTaken {
    repo: Arc<dyn UserRepository>,
}

impl EmailNotTaken {
    pub const CODE: RuleCode = RuleCode("EMAIL_TAKEN");

    pub fn new(repo: Arc<dyn UserRepository>) -> Self {
        Self { repo }
    }
}

impl AsyncBusinessLogicValidation<Email> for EmailNotTaken {
    fn validate<'a>(&'a self, email: &'a Email) -> ValidationFuture<'a> {
        Box::pin(async move {
            // A storage failure is not a broken rule: report it as `CheckFailed`.
            let taken =
self.repo.exists_by_email(email).await.map_err(AsyncValidationError::check_failed)?;
            if taken {
                Err(BusinessRuleError::new(Self::CODE, "the email is already registered")
                    .with_detail("email", email.as_str())
                    .into())
            } else {
                Ok(())
            }
        })
    }
}
```

El orden en el caso de uso, primero lo barato y al final lo que hace I/O:

```rust
// fragmento de use_cases/register_user.rs
start_validations(&[&AdultOnly, &TermsAccepted], &command)?;          // corta en la primera falla
start_validations_async(&[&self.email_not_taken], &email).await?;     // solo si las anteriores pasaron
```

Para formularios donde el usuario tiene que ver **todos** los errores juntos, `collect_validations` corre todas las reglas sincrónicas y devuelve un `Vec<BusinessRuleError>`.

Cómo llega cada error al cliente:

Resultado	REST (axum)	gRPC (tonic)
`BusinessRuleError` (regla rota)	422 con el `code` de la regla	`FAILED_PRECONDITION`
`AsyncValidationError::CheckFailed`	503	`UNAVAILABLE`

Reglas del equipo. Una regla por struct, con nombre de negocio (`AdultOnly`, no `Validation3`). Los adaptadores deciden por `code`, nunca por `message`. Las reglas no hacen trabajo de otros: una regla no guarda nada ni modifica estado.

Checkpoint. Escribí la regla `EmailDomainAllowed` que rechace emails de un dominio bloqueado, recibido en el constructor (usá `PredicateValidation` con una closure que capture la lista, o un struct propio). Agregala al caso de uso y a sus tests.

6.5 PostgreSQL con `tokio-postgres` y `deadpool-postgres`

Qué es. `tokio-postgres` es un cliente asincrónico de PostgreSQL. No es un ORM: escribís SQL, y vos decidís cómo mapear filas a tipos. No trae pool de conexiones, así que lo combinamos con `deadpool-postgres`.

El esquema (`migrations/0001_create_users.sql`):

▸ migrations/0001_create_users.sql

```
CREATE SEQUENCE IF NOT EXISTS users_id_seq;

CREATE TABLE IF NOT EXISTS users (
    id    BIGINT PRIMARY KEY DEFAULT nextval('users_id_seq'),
    email TEXT   NOT NULL UNIQUE
);
```

El pool, creado una vez en `main`:

```
// fragmento de main.rs (crate infrastructure)
let mut config = Config::new();
config.url = Some(database_url);
let pool = config.create_pool(Some(Runtime::Tokio1), NoTls)?;
let users: Arc<dyn UserRepository> = Arc::new(persistence::PgUserRepository::new(pool));
```

El repositorio implementa el puerto de `application`. Cada error de la base se convierte en `RepositoryError` para que ningún tipo de tokio-postgres cruce hacia adentro:

▸ crates/infrastructure/src/persistence/pg_user_repository.rs

```rust
use application::ports::{RepoFuture, UserRepository};
use deadpool_postgres::Pool;
use domain::entities::User;
use domain::value_objects::{Email, UserId};

use super::storage_error::storage_error;

/// `UserRepository` backed by PostgreSQL through a connection pool.
pub struct PgUserRepository {
    pool: Pool,
}

impl PgUserRepository {
    pub fn new(pool: Pool) -> Self {
        Self { pool }
    }
}

impl UserRepository for PgUserRepository {
    fn exists_by_email<'a>(&'a self, email: &'a Email) -> RepoFuture<'a, bool> {
        Box::pin(async move {
            let client = self.pool.get().await.map_err(storage_error)?;
            let row = client
                .query_one("SELECT EXISTS (SELECT 1 FROM users WHERE email = $1)", &[&email.as_str()])
                .await
                .map_err(storage_error)?;
            let exists: bool = row.get(0);
            Ok(exists)
        })
    }

    fn next_id(&self) -> RepoFuture<'_, UserId> {
        Box::pin(async move {
            let client = self.pool.get().await.map_err(storage_error)?;
            let row = client.query_one("SELECT nextval('users_id_seq')", &
[]).await.map_err(storage_error)?;
            let id: i64 = row.get(0);
            u64::try_from(id).map(UserId).map_err(storage_error)
        })
    }

    fn save<'a>(&'a self, user: &'a User) -> RepoFuture<'a, ()> {
        Box::pin(async move {
            let client = self.pool.get().await.map_err(storage_error)?;
            let id = i64::try_from(user.id.0).map_err(storage_error)?;
            client
                .execute("INSERT INTO users (id, email) VALUES ($1, $2)", &[&id, &user.email.as_str()])
                .await
                .map_err(storage_error)?;
            Ok(())
        })
    }
}
```

SINTAXIS Y API

- `self.pool.get().await` toma una conexión del pool; vuelve sola al pool cuando la variable sale de alcance (RAII, 3.5).
- `query_one(sql, &[&param1, &param2])` ejecuta y espera exactamente una fila; también existen `query` (varias), `query_opt` (cero o una) y `execute` (cantidad de filas afectadas).
- Los parámetros van como `$1`, `$2`. **Nunca** se arma SQL con `format!`: es la puerta a la inyección SQL.
- `let exists: bool = row.get(0);` lee la columna 0 y la convierte al tipo anotado. Si el tipo no coincide con el de la columna, hace panic: anotá el tipo correcto (`BIGINT` → `i64`, `TEXT` → `String` o `&str`, `BOOLEAN` → `bool`).

Transacciones. `Transaction` hace **rollback automático** si se destruye sin `commit` (lo vimos como patrón RAII en 3.5):

```rust
// esquema de uso dentro de un repositorio
let mut client = self.pool.get().await.map_err(storage_error)?;
let transaction = client.transaction().await.map_err(storage_error)?;
transaction.execute("UPDATE ...", &[&id]).await.map_err(storage_error)?;
transaction.execute("INSERT ...", &[&id]).await.map_err(storage_error)?;
transaction.commit().await.map_err(storage_error)?; // si algo falló antes, `?` sale y hay rollback
```

`transaction()` toma prestado el cliente de forma exclusiva (`&mut`). Si un caso de uso escribe con varios repositorios y todo tiene que ir en una sola transacción, el diseño tiene que decir cómo se comparte (por ejemplo, un puerto *unit of work*). EGM lo pregunta en la revisión.

Reglas del equipo.

- Pool con `deadpool-postgres`; una conexión por operación, devuelta al pool lo antes posible.
- Solo parámetros `$n`. Activá solo las features de tipos que uses (`with-uuid-1`, `with-chrono-0_4`, `with-serde_json-1`).
- TLS con `postgres-native-tls` o `postgres-openssl` en producción; `NoTls` solo en desarrollo local o en redes privadas, y el diseño lo justifica.
- Las migraciones viven en `migrations/`; la herramienta que las aplica la define cada diseño.

Errores comunes. Mapear `BIGINT` a `u64` (PostgreSQL no tiene enteros sin signo: es `i64`, y se convierte con `try_from`). Confiar solo en el chequeo previo de duplicados: dos pedidos simultáneos pueden pasar los dos; la restricción `UNIQUE` es la garantía final.

Checkpoint. Levantá PostgreSQL con `docker run --name pg -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres`, aplicá la migración con `psql`, y corré el servicio con `DATABASE_URL=postgres://postgres:postgres@localhost/postgres cargo run -p infrastructure`.

6.6 REST con axum

Qué es. axum es el framework HTTP del ecosistema Tokio. No tiene "magia": un handler es una función `async` cuyos parámetros son *extractores* (piezas del pedido) y cuyo resultado implementa `IntoResponse`. Los middlewares vienen de `tower` y `tower-http`.

El router y el estado compartido:

▸ crates/infrastructure/src/http/app_state.rs

```rust
use std::sync::Arc;

use application::use_cases::RegisterUserUseCase;

/// State shared by the HTTP handlers: one field per use case the API exposes.
#[derive(Clone)]
pub struct AppState {
    pub register_user: Arc<RegisterUserUseCase>,
}
```

▸ crates/infrastructure/src/http/router.rs

```rust
use axum::routing::post;
use axum::Router;

use super::register_user_handler::register_user;
use super::AppState;

/// Routes of the REST API.
pub fn router(state: AppState) -> Router {
    Router::new().route("/users", post(register_user)).with_state(state)
}
```

El handler, que solo traduce:

▸ crates/infrastructure/src/http/register_user_handler.rs

```rust
use axum::extract::State;
use axum::http::StatusCode;
use axum::Json;

use super::api_error::ApiError;
use super::dto::{RegisterUserRequest, UserResponse};
use super::AppState;

/// `POST /users`: translates HTTP to the use case and back. No business logic here.
pub async fn register_user(
    State(state): State<AppState>,
    Json(request): Json<RegisterUserRequest>, // body extractor goes last
) -> Result<(StatusCode, Json<UserResponse>), ApiError> {
    let user = state.register_user.execute(request.into()).await?;
    Ok((StatusCode::CREATED, Json(user.into())))
}
```

Los DTO separan el formato JSON de los tipos de `application`:

▸ crates/infrastructure/src/http/dto/register_user_request.rs

```rust
use application::commands::RegisterUserCommand;
use serde::Deserialize;

/// JSON body of `POST /users`.
#[derive(Debug, Deserialize)]
pub struct RegisterUserRequest {
    pub email: String,
    pub age: u8,
    pub accepted_terms: bool,
}

impl From<RegisterUserRequest> for RegisterUserCommand {
    fn from(request: RegisterUserRequest) -> Self {
        RegisterUserCommand { email: request.email, age: request.age, accepted_terms:
request.accepted_terms }
    }
}
```

El mapeo de errores a HTTP (el `@ControllerAdvice` de este servicio):

▸ crates/infrastructure/src/http/api_error.rs

```rust
use application::errors::RegisterUserError;
use axum::http::StatusCode;
use axum::response::{IntoResponse, Response};
use axum::Json;
use serde::Serialize;

/// HTTP translation of application errors. Clients decide by `code`, never by `message`.
pub struct ApiError(RegisterUserError);

#[derive(Serialize)]
struct ErrorBody {
    code: String,
    message: String,
}

impl From<RegisterUserError> for ApiError {
    fn from(error: RegisterUserError) -> Self {
        Self(error)
    }
}

impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (status, code, message) = match &self.0 {
            RegisterUserError::InvalidInput(e) => {
                (StatusCode::UNPROCESSABLE_ENTITY, "INVALID_INPUT".to_string(), e.to_string())
            }
            RegisterUserError::BusinessRule(rule) => {
                (StatusCode::UNPROCESSABLE_ENTITY, rule.code.to_string(), rule.message.clone())
            }
            RegisterUserError::RuleCheckFailed(_) => {
                (StatusCode::SERVICE_UNAVAILABLE, "UNAVAILABLE".to_string(), "try again
later".to_string())
            }
            RegisterUserError::Repository(_) => {
                (StatusCode::INTERNAL_SERVER_ERROR, "INTERNAL".to_string(), "internal
error".to_string())
            }
        };
        if status.is_server_error() {
            eprintln!("request failed: {}", self.0); // replace with `tracing::error!` in a real
service
        }
        (status, Json(ErrorBody { code, message })).into_response()
    }
}
```

SINTAXIS Y API

- `State(state): State<AppState>` es un patrón en el parámetro: extrae el estado y lo desarma en la misma línea. Lo mismo con `Json(request)`.
- El extractor que consume el cuerpo (`Json`, `String`) va **último**; axum lo exige en compilación.
- Las rutas con parámetros usan llaves: `"/users/{id}"`, y se extraen con `Path(id): Path<u64>`.
- `ApiError` envuelve a `RegisterUserError` porque la *orphan rule* no deja implementar un trait ajeno (`IntoResponse`) para un tipo ajeno (1.9).
- `match` sobre el error sin `_ =>`: si mañana `RegisterUserError` suma una variante, este archivo deja de compilar hasta decidir su código HTTP.

Reglas del equipo. `State` y no `Extension` (si falta un `Extension`, axum responde 500 en ejecución; si falta un `State`, no compila). El estado es `Clone` y barato de clonar (`Arc` adentro). Middlewares de `tower-http` para trazas, timeouts y CORS. Un handler por archivo.

Errores comunes. Poner lógica de negocio en el handler. Responder con el `message` interno de un error 500.

Checkpoint. Con el servicio corriendo, probá los tres casos:

```
curl -i -X POST localhost:8080/users -H 'content-type: application/json' -d
'{"email":"ana@example.com","age":30,"accepted_terms":true}'   → 201
(el mismo pedido otra vez)
→ 422 EMAIL_TAKEN
curl -i -X POST localhost:8080/users -H 'content-type: application/json' -d
'{"email":"ana","age":30,"accepted_terms":true}'           → 422 INVALID_INPUT
```

Después agregá `GET /users/{id}` con su caso de uso, su método en el puerto y su handler.

6.7 gRPC con tonic y el crate `proto`

Qué es. gRPC es RPC sobre HTTP/2 con mensajes Protocol Buffers. El contrato se escribe en un archivo `.proto`, y de él se genera el código del servidor y del cliente. En Rust usamos **tonic** (el framework) y **prost** (los mensajes). Lo elegimos en lugar de REST cuando los clientes son otros servicios, cuando importa el contrato tipado o cuando hace falta *streaming*.

Por qué un crate `proto` aparte: el contrato es compartido (servidor, clientes, otros servicios) y su código generado no tiene que mezclarse con la lógica. Solo `infrastructure` y los clientes dependen de él; los tipos de `prost` nunca llegan a `application` ni a `domain`.

El contrato:

▸ crates/proto/proto/users/v1/users.proto

```
syntax = "proto3";

package users.v1;

// Public gRPC contract of the users service.
// Compatibility rules: never reuse or renumber a field; mark removed fields as `reserved`.
service UserService {
  rpc RegisterUser(RegisterUserRequest) returns (RegisterUserResponse);
}

message RegisterUserRequest {
  string email = 1;
  uint32 age = 2;
  bool accepted_terms = 3;
}

message RegisterUserResponse {
  uint64 id = 1;
  string email = 2;
}
```

La generación de código corre en `build.rs` y necesita `protoc` instalado:

▸ crates/proto/build.rs

```rust
//! Generates the Rust code for the `.proto` files. Requires `protoc` on the build machine.

fn main() -> Result<(), Box<dyn std::error::Error>> {
    tonic_prost_build::compile_protos("proto/users/v1/users.proto")?;
    Ok(())
}
```

▸ crates/proto/src/lib.rs

```rust
//! Generated gRPC contract. No hand-written logic lives here.

pub mod users {
    pub mod v1 {
        tonic::include_proto!("users.v1");
    }
}
```

El servicio, en `infrastructure`, traduce mensajes protobuf al caso de uso y vuelta:

▸ crates/infrastructure/src/grpc/grpc_user_service.rs

```rust
use std::sync::Arc;

use application::commands::RegisterUserCommand;
use application::use_cases::RegisterUserUseCase;
use proto::users::v1::user_service_server::UserService;
use proto::users::v1::{RegisterUserRequest, RegisterUserResponse};
use tonic::{Request, Response, Status};

use super::status_mapping::status_from;

/// gRPC adapter: translates protobuf messages to the use case and back.
pub struct GrpcUserService {
    register_user: Arc<RegisterUserUseCase>,
}

impl GrpcUserService {
    pub fn new(register_user: Arc<RegisterUserUseCase>) -> Self {
        Self { register_user }
    }
}

#[tonic::async_trait]
impl UserService for GrpcUserService {
    async fn register_user(
        &self,
        request: Request<RegisterUserRequest>,
    ) -> Result<Response<RegisterUserResponse>, Status> {
        let message = request.into_inner();
        let age = u8::try_from(message.age).map_err(|_| Status::invalid_argument("age is out of
range"))?;
        let command = RegisterUserCommand { email: message.email, age, accepted_terms:
message.accepted_terms };

        let user = self.register_user.execute(command).await.map_err(status_from)?;
        Ok(Response::new(RegisterUserResponse { id: user.id.0, email: user.email.as_str().to_string()
}))
    }
}
```

▸ crates/infrastructure/src/grpc/status_mapping.rs

```rust
use application::errors::RegisterUserError;
use tonic::Status;

/// gRPC translation of application errors. A function, not `From`: both types are foreign
/// to this crate, so the orphan rule forbids `impl From<RegisterUserError> for Status` here.
pub(crate) fn status_from(error: RegisterUserError) -> Status {
    match error {
        RegisterUserError::InvalidInput(e) => Status::invalid_argument(e.to_string()),
        RegisterUserError::BusinessRule(rule) => Status::failed_precondition(rule.to_string()),
        RegisterUserError::RuleCheckFailed(_) => Status::unavailable("a business rule could not be
checked"),
        RegisterUserError::Repository(_) => Status::internal("internal error"),
    }
}
```

REST y gRPC en el mismo binario, con dos puertos:

▸ crates/infrastructure/src/main.rs

```rust
//! Composition root: builds the infrastructure, the use cases, and the adapters, then serves REST and
gRPC.

mod grpc;
mod http;
mod persistence;

use std::net::SocketAddr;
use std::sync::Arc;

use anyhow::Context;
use application::ports::UserRepository;
use application::use_cases::RegisterUserUseCase;
use deadpool_postgres::{Config, Runtime};
use proto::users::v1::user_service_server::UserServiceServer;
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // Configuration
    let database_url = std::env::var("DATABASE_URL").context("DATABASE_URL is not set")?;
    let http_addr: SocketAddr = "0.0.0.0:8080".parse()?;
    let grpc_addr: SocketAddr = "0.0.0.0:50051".parse()?;

    // Infrastructure: a connection pool (tokio-postgres has none of its own)
    let mut config = Config::new();
    config.url = Some(database_url);
    let pool = config.create_pool(Some(Runtime::Tokio1), NoTls)?;
    let users: Arc<dyn UserRepository> = Arc::new(persistence::PgUserRepository::new(pool));

    // Use cases
    let register_user = Arc::new(RegisterUserUseCase::new(users));

    // Adapters: REST and gRPC on separate listeners
    let rest = http::router(http::AppState { register_user: Arc::clone(&register_user) });
    let grpc_service = UserServiceServer::new(grpc::GrpcUserService::new(register_user));

    let listener = tokio::net::TcpListener::bind(http_addr).await?;
    let rest_server = async { axum::serve(listener, rest).await.context("REST server failed") };
    let grpc_server = async {
        tonic::transport::Server::builder()
            .add_service(grpc_service)
            .serve(grpc_addr)
            .await
            .context("gRPC server failed")
    };

    println!("REST on http://{http_addr}, gRPC on {grpc_addr}");
    tokio::try_join!(rest_server, grpc_server)?;
    Ok(())
}
```

SINTAXIS Y API

- `tonic::include_proto!("users.v1")` incluye el código que generó `build.rs` para ese paquete.
- `#[tonic::async_trait]` hace falta porque el trait generado tiene métodos `async` y tonic lo usa como objeto dinámico.
- El código generado nombra los métodos en `snake_case`: `rpc RegisterUser` → `fn register_user`.
- `tokio::try_join!` espera los dos servidores y termina si cualquiera falla.
- tonic también puede servir en el mismo puerto que axum (`tonic::service::Routes::into_axum_router`); el diseño de cada servicio decide.

Reglas del equipo.

- El `.proto` es un contrato público: nunca reutilices ni renumeres un campo; marcá los campos eliminados como `reserved`. Versioná el paquete (`users.v1`).
- Los códigos `Status` siguen el significado de gRPC: `INVALID_ARGUMENT` para entrada mal formada, `FAILED_PRECONDITION` para una regla de negocio, `UNAVAILABLE` para algo transitorio, `INTERNAL` para el resto.
- CI instala `protoc` antes de compilar.

Errores comunes. Usar los mensajes generados como tipos de dominio. Olvidar que `uint32` llega como `u32` (en el ejemplo, la edad se valida al convertirla a `u8`).

Checkpoint. Con el servicio corriendo, llamalo con grpcurl usando el `.proto` (sin reflexión):

```
grpcurl -plaintext -import-path crates/proto/proto -proto users/v1/users.proto \
  -d '{"email":"luz@example.com","age":25,"accepted_terms":true}' \
  localhost:50051 users.v1.UserService/RegisterUser
```

Después agregá el `rpc GetUser` al contrato y su implementación.

6.8 Escritorio y móvil con Tauri 2

Qué es. Tauri arma aplicaciones de escritorio (Windows, macOS, Linux) y móviles (iOS, Android) con la interfaz en web (HTML, CSS, JavaScript o un framework) y la lógica en Rust. Usa el *webview* del sistema operativo en lugar de empaquetar un navegador, así que los binarios son chicos.

Cómo se conecta con nuestra arquitectura. La app es un crate más (`src-tauri`). Sus **comandos** son adaptadores: reciben datos del frontend, llaman a un caso de uso de `application` y devuelven el resultado. Se crea con `cargo create-tauri-app` (o `npm create tauri-app`).

```rust
// src-tauri/src/commands.rs
use std::sync::Arc;

use application::commands::RegisterUserCommand;
use application::errors::RegisterUserError;
use application::use_cases::RegisterUserUseCase;
use serde::Serialize;

/// Alias del estado administrado: un tipo equivocado en `State<T>` compila y falla en ejecución.
pub type RegisterUser = Arc<RegisterUserUseCase>;

/// Error que recibe el frontend como `{ kind, message }`.
#[derive(Debug, Serialize)]
#[serde(tag = "kind", content = "message")]
pub enum CommandError {
    InvalidInput(String),
    BusinessRule(String),
    Unavailable(String),
}

impl From<RegisterUserError> for CommandError {
    fn from(error: RegisterUserError) -> Self {
        match error {
            RegisterUserError::InvalidInput(e) => CommandError::InvalidInput(e.to_string()),
            RegisterUserError::BusinessRule(rule) => CommandError::BusinessRule(rule.code.to_string()),
            other => CommandError::Unavailable(other.to_string()),
        }
    }
}

/// Comando: traduce, llama al caso de uso y devuelve el id.
#[tauri::command]
pub async fn register_user(
    register_user: tauri::State<'_, RegisterUser>,
    email: String,
    age: u8,
    accepted_terms: bool,
) -> Result<u64, CommandError> {
    let user = register_user.execute(RegisterUserCommand { email, age, accepted_terms }).await?;
    Ok(user.id.0)
}
```

```rust
// src-tauri/src/lib.rs
mod commands;

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let register_user: commands::RegisterUser = build_register_user(); // armado como en main.rs
    tauri::Builder::default()
        .manage(register_user)
        .invoke_handler(tauri::generate_handler![commands::register_user])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

Desde el frontend (JavaScript o TypeScript), los nombres de los argumentos van en *camelCase*:

```ts
import { invoke } from '@tauri-apps/api/core';
const id = await invoke('register_user', { email, age: 30, acceptedTerms: true });
```

REGLAS DE TAURI QUE CONVIENE SABER DE MEMORIA

- Los comandos viven en un módulo `commands` y son `pub` ahí. En `lib.rs` no pueden ser `pub`, por una limitación de la generación de código.
- Todos se registran en **un solo** `generate_handler!`: si llamás `invoke_handler` dos veces, solo vale la última.
- Un comando `async` con argumentos prestados (`&str`, `State<'_, T>`) tiene que devolver `Result`.
- Todo lo que devuelve un comando, **incluidos los errores**, implementa `serde::Serialize`.
- El estado registrado con `manage` no necesita `Arc` para Tauri; acá lo usamos porque el caso de uso ya se comparte como `Arc`.
- **Seguridad:** las *capabilities* (`src-tauri/capabilities/*.json`) dan a cada ventana solo los permisos que necesita. Los comandos propios están habilitados para todas las ventanas salvo que los restrinjas en `build.rs`.

Errores comunes. Poner reglas de negocio en el comando. Usar el `State<T>` con un tipo distinto del registrado (falla recién al invocar). Dar permisos amplios "para probar" y dejarlos.

Checkpoint. Creá una app con `cargo create-tauri-app`, agregá el comando `register_user` usando un repositorio en memoria, y llamalo desde un formulario del frontend mostrando el `kind` del error cuando falle.

6.9 Interfaces en Rust con Dioxus 0.7

Qué es. Dioxus es un framework de interfaces escrito en Rust, con un modelo parecido a React: componentes, estado reactivo y una sintaxis declarativa (`rsx!`). El mismo código corre en web (WebAssembly), escritorio y móvil. En modo **fullstack**, las *server functions* corren en el servidor y el cliente las llama como funciones normales.

Lo esencial:

```rust
use dioxus::prelude::*;

#[component]
fn Contador(inicial: i32) -> Element {
    let mut cuenta = use_signal(|| inicial);        // estado reactivo
    let doble = use_memo(move || cuenta() * 2);     // estado derivado

    rsx! {
        p { "Cuenta: {cuenta} (doble: {doble})" }
        button { onclick: move |_| cuenta += 1, "Sumar" }
    }
}
```

SINTAXIS

- `#[component]` convierte una función en componente; sus parámetros son las *props*.
- `use_signal` crea estado; leerlo (`cuenta()`) suscribe al componente y escribirlo (`cuenta += 1` , `.set(...)`) lo vuelve a renderizar.
- `use_memo` calcula estado derivado; `use_effect` , efectos secundarios; `use_resource` y `use_action` , trabajo asincrónico.
- `rsx!` describe la interfaz: elementos, atributos, texto con interpolación `{variable}` y manejadores `onclick: move |_| ...` .

Una server function que llama a nuestro caso de uso. El código del servidor queda detrás de la feature `server` , para que la base de datos y `application` no lleguen al paquete web:

```rust
use dioxus::fullstack::{extract::State, FullstackContext};
use dioxus::prelude::*;

/// Servicios del servidor. Se registran como `Extension` en `main` (ejemplo oficial
`server_state.rs`).
#[cfg(feature = "server")]
#[derive(Clone)]
struct AppServices {
    register_user: std::sync::Arc<application::use_cases::RegisterUserUseCase>,
}

#[cfg(feature = "server")]
impl dioxus::server::axum::extract::FromRef<FullstackContext> for AppServices {
    fn from_ref(context: &FullstackContext) -> Self {
        context.extension::<AppServices>().expect("AppServices is registered in main")
    }
}

#[post("/api/users", services: State<AppServices>)]
async fn register_user(email: String, age: u8, accepted_terms: bool) -> Result<u64, ServerFnError> {
    let command = application::commands::RegisterUserCommand { email, age, accepted_terms };
    match services.register_user.execute(command).await {
        Ok(user) => Ok(user.id.0),
        Err(application::errors::RegisterUserError::BusinessRule(rule)) => {
            Err(ServerFnError::ServerError { message: rule.code.to_string(), details: None, code: 422
})
        }
        Err(other) => Err(ServerFnError::new(other)),
    }
}
```

Y un componente que la usa:

```rust
fn Registro() -> Element {
    let mut email = use_signal(String::new);
    let mut registrar = use_action(register_user);

    rsx! {
        input { value: "{email}", oninput: move |e| email.set(e.value()) }
        button { onclick: move |_| { registrar.call(email(), 30, true); }, "Registrarme" }
        pre { "{registrar.value():?}" }
    }
}
```

Cargo.toml de una app fullstack:

```toml
[dependencies]
dioxus = { version = "0.7", features = ["fullstack"] }
application = { workspace = true, optional = true }  # solo en el servidor
tokio = { workspace = true, optional = true }

[features]
web = ["dioxus/web"]
desktop = ["dioxus/desktop"]
server = ["dioxus/server", "dep:application", "dep:tokio"]
```

Reglas del equipo, tomadas de la guía de antipatrones de Dioxus:

- Claves estables y únicas en las listas (no el índice).
- Nada de `Mutex` o `RefCell` en las props: pasá `Signal` o datos inmutables.
- No escribas estado durante el render; usá `use_effect` o `use_memo`.
- Estado en piezas chicas, no un struct gigante.
- Código no determinista (aleatorio, hora) dentro de un hook, no en el cuerpo del componente.
- `PartialEq` de las props derivado, nunca "siempre igual".

Errores comunes. Importar `application` o `tokio` sin la feature `server` (el paquete web no compila). Poner reglas de negocio en la server function.

Checkpoint. Creá una app con `dx new` (plantilla fullstack), agregá la server function `register_user` con un repositorio en memoria y corrila con `dx serve`.

Parte 7 · Trabajar con EGM

7.1 Instalar nuestro EGM

EGM (Elephant-Goldfish Model) es el plugin con el que diseñamos y revisamos cada funcionalidad antes de programarla. Usamos **nuestro fork**, que tiene el perfil Rust: si instalás el original de Google, no vas a tener el revisor ni el estándar de Rust.

```
Claude Code:
  claude plugin marketplace add mdambro/elephant-goldfish-model
  claude plugin install egm@egm

Codex:
  codex plugin marketplace add mdambro/elephant-goldfish-model
  codex plugin add egm@egm
```

Abrí una sesión nueva después de instalarlo. En Claude Code los comandos llevan el prefijo `/egm:` (por ejemplo `/egm:egm-draft`).

7.2 La idea: un elefante, peces y una persona

- **Elefante:** un agente con todo el contexto del proyecto, que explora el problema con vos y redacta el diseño.
- **Peces (Goldfish):** revisores que leen el diseño **sin** la conversación previa. Como no saben lo que "ya hablaron", encuentran suposiciones no escritas, huecos y riesgos.
- **Vos:** decidís los trade-offs, aprobás el diseño y sos responsable del resultado. La IA es una herramienta, no quien decide.

El flujo habitual:

Paso	Comando	Resultado
1. Explorar la idea	`/egm-brainstorm`	`concepts_brief.md`
2. Redactar el diseño	`/egm-draft`	`docs/designs/<funcionalidad>.md`, sección por sección
3. Revisarlo	`/egm-review <diseño>`	Informes de revisión por etapa
4. Resolver hallazgos	`/egm-resolve` (o desde la revisión)	Diseño corregido
5. Revisión humana	PR con el diseño	Diseño aprobado
6. Implementar	Pedirle al agente que implemente el diseño	Código que respeta la lista de archivos del plan
7. Mantenerlo vivo	`/egm-drift-report` (semanal)	Diferencias entre diseño y código

No todo pasa por EGM: un bug chico y claro se arregla directamente.

7.3 Qué agrega el perfil Rust

- **Rust por defecto.** El diseño registra `Language: Rust (default)` salvo que digas otro lenguaje.
- **Sección "Rust Design Decisions"** en el plan técnico: crates y capas, transporte (REST, gRPC, Tauri, Dioxus), responsabilidades, persistencia, validación, errores, polimorfismo, ownership y concurrencia, dependencias. Cuando una decisión tiene más de una respuesta válida (por ejemplo `enum` o trait), EGM te presenta las opciones y vos elegís.
- **El plan de implementación en Rust:** cambios de `Cargo.toml`, firmas en sintaxis Rust, variantes de error, derives, tests y los comandos de verificación (`cargo fmt --check`, `cargo clippy ... -D warnings`, `cargo test`).
- **El revisor `egm-rust`** en las etapas 7 (diseño) y 9 (plan) de `/egm-review`. Aplica el **Rust Standard** del plugin, que resume esta guía: decisiones abiertas, estados inválidos representables, antipatrones (Parte 4), patrones usados sin un problema que los justifique, tipos con más de una responsabilidad, dependencias que apuntan hacia afuera y desvíos del stack sin razón escrita.

Los revisores no leen el `AGENTS.md` ni el `CLAUDE.md` del proyecto: solo ven el diseño, los estándares del plugin y los archivos que les pases como contexto. Lo que el equipo quiere que se cumpla tiene que estar en el estándar o en el diseño.

7.4 Un ciclo completo, con el proyecto de inducción

Supongamos la funcionalidad "consultar un usuario por id" (`GET /users/{id}` y `rpc GetUser`).

1. **Brainstorm:** `/egm:egm-brainstorm Agregar la consulta de un usuario por id, por REST y gRPC`. El agente te hace preguntas de a una, y seis revisores cuestionan la idea (¿hace falta paginar? ¿qué pasa si no existe?).
2. **Draft:** `/egm:egm-draft`. Vas aprobando cada sección. En "Rust Design Decisions" vas a decidir, por ejemplo, si "no existe" es un `Option` en el puerto o una variante de error.
3. **Review:** `/egm:egm-review docs/designs/get_user.md`. En la etapa 9, `egm-rust` va a pedir, entre otras cosas, la firma exacta del método nuevo del puerto y la fila de la tabla de mapeo de errores para "no existe" (404 y `NOT_FOUND`).
4. **Implementación:** con el diseño aprobado, pedile al agente que lo implemente. El plan dice qué archivos se crean o modifican; nada más.

7.5 El contexto del proyecto

Cada repositorio tiene un `AGENTS.md` (o `CLAUDE.md`) con las convenciones, que lee el agente que redacta e implementa. Un punto de partida:

```
## Rust Conventions

- Language: Rust, edition 2021, MSRV <version>.
- Workspace: crates/domain (no I/O), crates/application (use cases, ports,
  validations_rs rules), crates/infrastructure (axum or tonic adapters,
  tokio-postgres repositories, main), crates/proto (only with gRPC).
  Dependencies point inward only; prost types never leave infrastructure.
- One responsibility per type: one use case struct with one `execute`, one
  validation rule per struct, one file per public type.
- Errors: thiserror enums per crate; anyhow only in main.
- Async: tokio; ports are traits with Send + Sync.
- Database: tokio-postgres + deadpool-postgres; $1 parameters only.
- Lints: configured in [workspace.lints]; CI runs
  cargo fmt --check, cargo clippy --all-targets --all-features -- -D warnings,
  and cargo test --all-features.
- Follow the EGM Rust Standard for patterns, anti-patterns, and the stack.
```

Parte 8 · Ejercicios y proyecto final

8.1 Ejercicios del lenguaje

Cada ejercicio practica un patrón. Hacelos en el Playground o en un proyecto con `cargo new`.

1. Newtype para plata (3.1). Creá `Precio(u64)` en centavos. Implementá `Display` para que muestre `$ 1234,50`, e implementá el trait `std::ops::Add` para poder sumar dos precios con `+`. Bonus: que no se pueda construir un precio inválido desde un `&str` con `Precio::parse("12,50")`.

2. Del struct trucho al enum (2.3). Este struct permite estados absurdos:

```
struct Pedido {
    pagado: bool,
    enviado: bool,
    codigo_seguimiento: Option<String>,
    motivo_cancelacion: Option<String>,
}
```

Rediseñalo como enum para que los estados inválidos sean irrepresentables, y escribí un método `enviar(&mut self, codigo: String)` usando `mem::take` donde haga falta.

3. Builder de búsqueda (3.2). Hacé un `BusquedaEventos` con filtros opcionales: ciudad, rango de fechas (dos `String` por ahora) y precio máximo. `build()` tiene que fallar si la fecha "desde" es mayor que "hasta".

4. Strategy de comisiones (3.3 y 3.8). Definí el trait `Comision` con `fn calcular(&self, venta: u64) -> u64`. Implementalo para un struct `Porcentual(u8)`, para un struct `Fija(u64)` y, con una blanket impl, para cualquier closure `Fn(u64) -> u64`. Armá una función que elija la comisión en runtime según el canal de venta ("web", "app", "boletería") devolviendo `Box<dyn Comision>`.

5. Guarda RAII (2.6 y 3.5). Escribí un `MedidorDeCasoDeUso` que al crearse guarde el nombre del caso de uso y la hora de inicio, y al destruirse imprima cuánto tardó. Usalo dentro del `execute` de `RegisterUserUseCase` y comprobá que mide también cuando el caso de uso devuelve error.

6. Typestate de checkout (3.9). Modelá un `Checkout<E>` con los estados `Carrito`, `ConDatosDePago` y `Confirmado`, de forma que `confirmar()` solo exista después de cargar los datos de pago.

7. Cazar clones (4.1). ¿Cuáles de estos clones sobran?

```
fn largo(s: String) -> usize { s.len() }

fn main() {
    let nombre = String::from("Ferris");
    let n = largo(nombre.clone());
    println!("{nombre} tiene {n} letras");

    let lista = vec![1, 2, 3];
    let total: i32 = lista.clone().iter().sum();
    println!("{total} {lista:?}");
}
```

Pista: uno se arregla cambiando la firma de una función (2.1) y el otro se puede borrar directamente.

8.2 Proyecto final de inducción

Agregá al proyecto de inducción el caso de uso **"cambiar el email de un usuario"**, de punta a punta, siguiendo el flujo de la Parte 7:

1. **Diseño con EGM:** brainstorm, draft y review hasta que `egm-rust` no tenga hallazgos bloqueantes. Decisiones que vas a tener que tomar: qué pasa si el email nuevo ya existe, si el cambio va en una transacción, y cómo se informa "usuario inexistente".
2. **`domain` y `application`:** el caso de uso `ChangeEmailUseCase` (un struct, un `execute`), sus reglas con `validations_rs` (reutilizá `EmailNotTaken`) y su error con `thiserror`.
3. **`infrastructure`:** el método del repositorio en tokio-postgres, `PATCH /users/{id}/email` en axum y `rpc ChangeEmail` en el `.proto` y en tonic, con sus filas en los mapeos de errores.
4. **Tests:** unitarios con un repositorio en memoria (incluido el test de `Send`), y una prueba manual con `curl` y `grpcurl`.
5. **Verificación:** `cargo fmt --check`, `cargo clippy --all-targets --all-features -- -D warnings` y `cargo test` sin errores.
6. **PR** con el diseño y el código, revisado por alguien del equipo.

Bonus para el rol frontend: el mismo caso de uso desde una app Tauri o Dioxus.

Fuentes

- *Rust Design Patterns*, rust-unofficial: rust-unofficial.github.io/patterns.
- *The Rust Programming Language*, el libro oficial: doc.rust-lang.org/book.
- Tokio: docs.rs/tokio y tokio.rs/tokio/tutorial.
- axum: docs.rs/axum.
- tokio-postgres: docs.rs/tokio-postgres; deadpool-postgres: docs.rs/deadpool-postgres.
- tonic: github.com/hyperium/tonic (incluye tutoriales de *helloworld* y *routeguide*).
- Tauri 2: v2.tauri.app/es/start, comandos, estado, capabilities.
- Dioxus 0.7: dioxuslabs.com/learn/0.7, server functions, antipatrones.
- validations_rs: github.com/mdambro/validations_rs.
- EGM (nuestro fork): github.com/mdambro/elephant-goldfish-model.
- Tris Oaten, *Rust Data Modelling Without Classes* (No Boilerplate): youtube.com/watch?v=z-0-bbc80JM.
- Rust Playground: play.rust-lang.org.